

BASICS OF MACHINE LEARNING

UNIT: 1

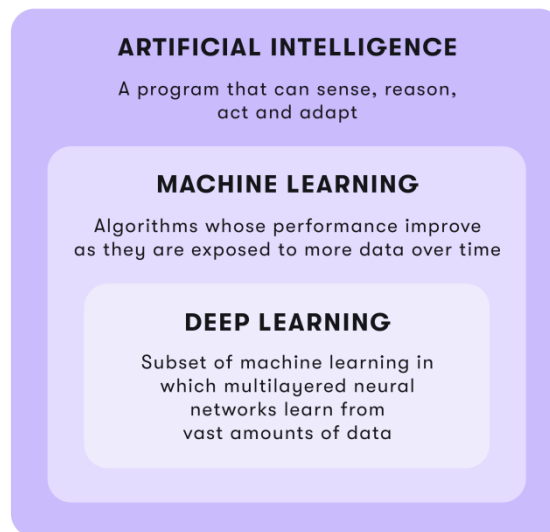
INTRODUCTION:

Arthur Samuel first used the term "machine learning" in 1959.

Machine Learning (ML) is a specialized branch of Artificial Intelligence (AI) that enables systems to learn from data and improve their performance over time without being explicitly programmed for each task. This capability allows machines to identify patterns, make predictions, and perform complex decision-making based on historical data.

A subset of artificial intelligence known as machine learning focuses primarily on the creation of algorithms that enable a computer to independently learn from data and previous experiences.

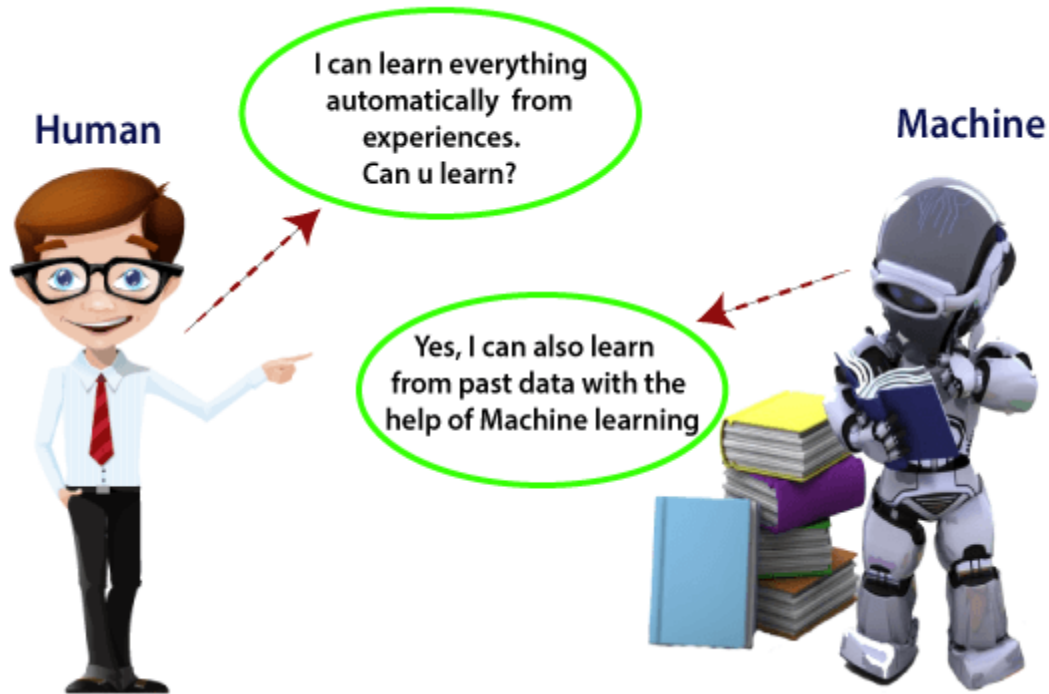
Without being explicitly programmed, machine learning enables a machine to automatically learn from data, improve performance from experiences, and predict things.



Definition and Functionality:

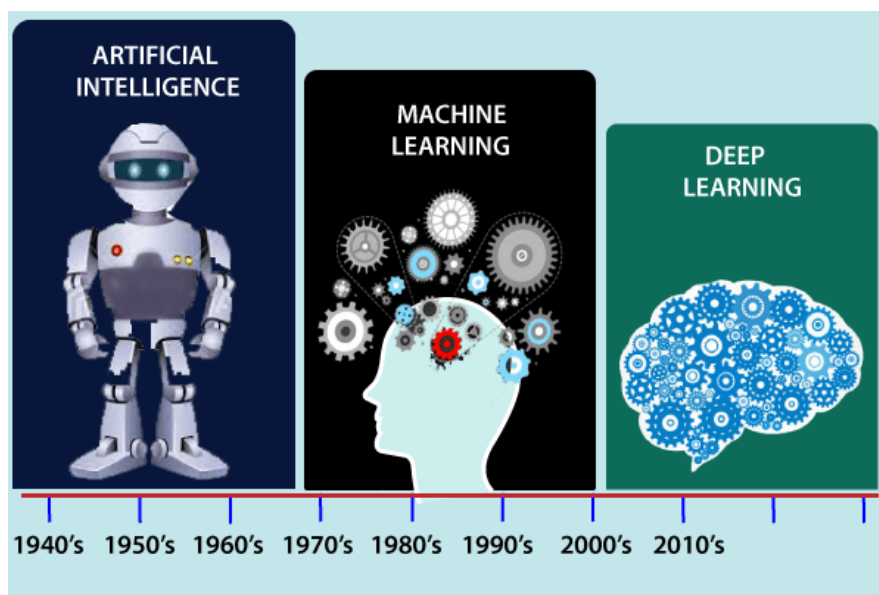
Machine Learning as AI: ML is fundamentally a capable of AI that focuses on developing algorithms capable of learning from and making predictions based on data. It operated by optimizing model parameters through calculations, allowing the model's behavior to reflect the data it has processes.

Learning Process: The learning process involves feeding large datasets into algorithms that can adjust their internal parameters iteratively. This allows the model to minimize prediction errors and improve accuracy as it encounters more data. For example, a machine learning system can be trained to distinguish between fraudulent and legitimate transactions by analyzing patterns in historical transaction data.



History:

The history of machine learning (ML) is rich and spans several decades, reflecting the evolution of ideas and technologies that have shaped the field. Before some years (about 40-50 years), machine learning was science fiction, but today it is the part of our daily life. Machine learning is making our day to day life easy from **self-driving cars** to **Amazon virtual assistant "Alexa"**. However, the idea behind machine learning is so old and has a long history. Below some milestones are given which have occurred in the history of machine learning:



The early history of Machine Learning (Pre-1940):

- **1834:** In 1834, Charles Babbage, the father of the computer, conceived a device that could be programmed with punch cards. However, the machine was never built, but all modern computers rely on its logical structure.
- **1936:** In 1936, Alan Turing gave a theory that how a machine can determine and execute a set of instructions.

The era of stored program computers:

- **1940:** In 1940, the first manually operated computer, "ENIAC" was invented, which was the first electronic general-purpose computer. After that stored program computer such as EDSAC in 1949 and EDVAC in 1951 were invented.
- **1943:** In 1943, a human neural network was modeled with an electrical circuit. In 1950, the scientists started applying their idea to work and analyzed how human neurons might work.

Computer machinery and intelligence:

- **1950:** In 1950, Alan Turing published a seminal paper, "**Computer Machinery and Intelligence**," on the topic of artificial intelligence. **In his paper, he asked, "Can machines think?"**
-

Machine intelligence in Games:

- **1952:** Arthur Samuel, who was the pioneer of machine learning, created a program that helped an IBM computer to play a checkers game. It performed better more it played.
 - **1959:** In 1959, the term "Machine Learning" was first coined by **Arthur Samuel**.
-

The first "AI" winter:

- The duration of 1974 to 1980 was the tough time for AI and ML researchers, and this duration was called as **AI winter**.
 - In this duration, failure of machine translation occurred, and people had reduced their interest from AI, which led to reduced funding by the government to the researches.
-

Machine Learning from theory to reality

- **1959:** In 1959, the first neural network was applied to a real-world problem to remove echoes over phone lines using an adaptive filter.
 - **1985:** In 1985, Terry Sejnowski and Charles Rosenberg invented a neural network **NETtalk**, which was able to teach itself how to correctly pronounce 20,000 words in one week.
 - **1997:** The IBM's **Deep blue** intelligent computer won the chess game against the chess expert Garry Kasparov, and it became the first computer which had beaten a human chess expert.
-

Machine Learning at 21st century

2006:

- Geoffrey Hinton and his group presented the idea of profound getting the hang of utilizing profound conviction organizations.
- The Elastic Compute Cloud (EC2) was launched by Amazon to provide scalable computing resources that made it easier to create and implement machine learning models.

2007:

- Participants were tasked with increasing the accuracy of Netflix's recommendation algorithm when the Netflix Prize competition began.
 - Support learning made critical progress when a group of specialists utilized it to prepare a PC to play backgammon at a top-notch level.
-

2008:

- Google delivered the Google Forecast Programming interface, a cloud-based help that permitted designers to integrate AI into their applications.
 - Confined Boltzmann Machines (RBMs), a kind of generative brain organization, acquired consideration for their capacity to demonstrate complex information conveyances.
-

2009:

- Profound learning gained ground as analysts showed its viability in different errands, including discourse acknowledgment and picture grouping.
 - The expression "Large Information" acquired ubiquity, featuring the difficulties and open doors related with taking care of huge datasets.
-

2010:

- The ImageNet Huge Scope Visual Acknowledgment Challenge (ILSVRC) was presented, driving progressions in PC vision, and prompting the advancement of profound convolutional brain organizations (CNNs).
-

2011:

- On Jeopardy! IBM's Watson defeated human champions., demonstrating the potential of question-answering systems and natural language processing.

2012:

- AlexNet, a profound CNN created by Alex Krizhevsky, won the ILSVRC, fundamentally further developing picture order precision and laying out profound advancing as a predominant methodology in PC vision.
- Google's Cerebrum project, drove by Andrew Ng and Jeff Dignitary, utilized profound figuring out how to prepare a brain organization to perceive felines from unlabeled YouTube recordings.

2013:

- Ian Goodfellow introduced generative adversarial networks (GANs), which made it possible to create realistic synthetic data.
- Google later acquired the startup DeepMind Technologies, which focused on deep learning and artificial intelligence.

2014:

- Facebook presented the DeepFace framework, which accomplished close human precision in facial acknowledgment.
- AlphaGo, a program created by DeepMind at Google, defeated a world champion Go player and demonstrated the potential of reinforcement learning in challenging games.

2015:

- Microsoft delivered the Mental Toolbox (previously known as CNTK), an open-source profound learning library.
-

- The performance of sequence-to-sequence models in tasks like machine translation was enhanced by the introduction of the idea of attention mechanisms.

2016:

- The goal of explainable AI, which focuses on making machine learning models easier to understand, received some attention.
- Google's DeepMind created AlphaGo Zero, which accomplished godlike Go abilities to play without human information, utilizing just support learning.

2017:

- Move learning acquired noticeable quality, permitting pretrained models to be utilized for different errands with restricted information.
 - Better synthesis and generation of complex data were made possible by the introduction of generative models like variational autoencoders (VAEs) and Wasserstein GANs.
 - These are only a portion of the eminent headways and achievements in AI during the predefined period. The field kept on advancing quickly past 2017, with new leap forwards, strategies, and applications arising.
-

Need for Machine Learning:

The demand for machine learning is steadily rising. Because it is able to perform tasks that are too complex for a person to directly implement, machine learning is required. Humans are constrained by our inability to manually access vast amounts of data; as a result, we require computer systems, which is where machine learning comes in to simplify our lives.

By providing them with a large amount of data and allowing them to automatically explore the data, build models, and predict the required output, we can train machine learning algorithms. The cost function can be used to determine the amount of data and the machine learning algorithm's performance. We can save both time and money by using machine learning.

The significance of AI can be handily perceived by its utilization's cases, Presently, AI is utilized in self-driving vehicles, digital misrepresentation identification, face acknowledgment, and companion idea by Facebook, and so on. Different top organizations, for example, Netflix and Amazon have constructed AI models that are utilizing an immense measure of information to examine the client interest and suggest item likewise.

Following are some key points which show the importance of Machine Learning:

- Rapid increment in the production of data
 - Solving complex problems, which are difficult for a human
 - Decision making in various sector including finance
 - Finding hidden patterns and extracting useful information from data.
-

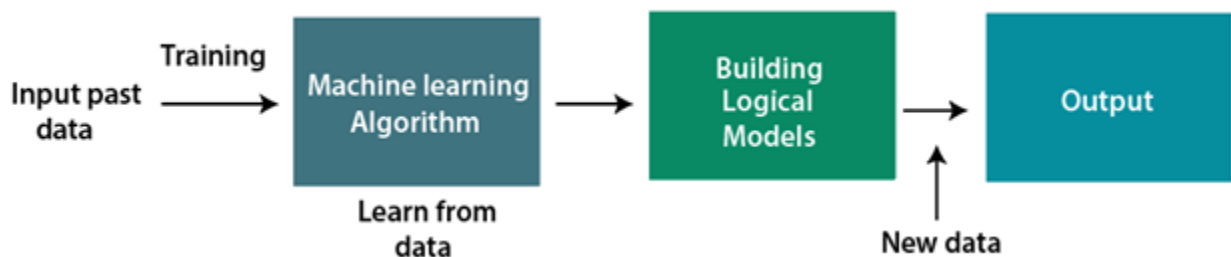
Features of Machine Learning:

- Machine learning uses data to detect various patterns in a given dataset.
 - It can learn from past data and improve automatically.
 - It is a data-driven technology.
 - Machine learning is much similar to data mining as it also deals with the huge amount of the data.
-

How does Machine Learning work:

A machine learning system builds prediction models, learns from previous data, and predicts the output of new data whenever it receives it. The amount of data helps to build a better model that accurately predicts the output, which in turn affects the accuracy of the predicted output.

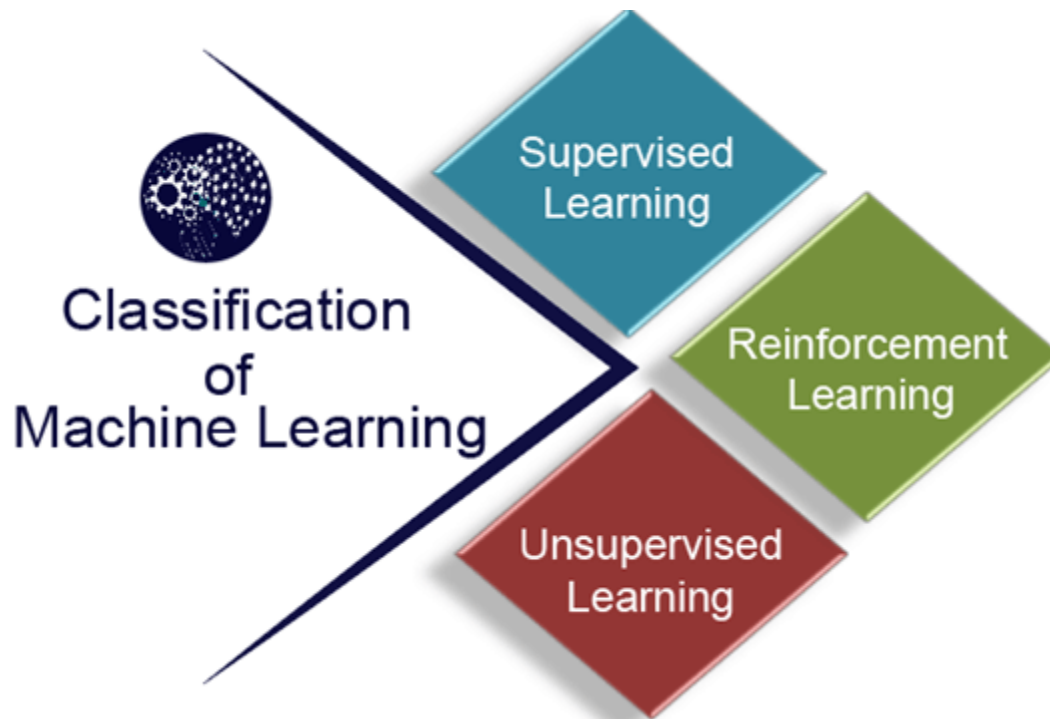
Let's say we have a complex problem in which we need to make predictions. Instead of writing code, we just need to feed the data to generic algorithms, which build the logic based on the data and predict the output. Our perspective on the issue has changed as a result of machine learning. The Machine Learning algorithm's operation is depicted in the following block diagram:



Classification of Machine Learning:

At a broad level, machine learning can be classified into three types:

1. **Supervised learning**
2. **Unsupervised learning**
3. **Reinforcement learning**



1) Supervised Learning

In supervised learning, sample labeled data are provided to the machine learning system for training, and the system then predicts the output based on the training data.

The system uses labeled data to build a model that understands the datasets and learns about each one. After the training and processing are done, we test the model with sample data to see if it can accurately predict the output.

The mapping of the input data to the output data is the objective of supervised learning. The managed learning depends on oversight, and it is equivalent to when an understudy learns things in the management of the educator. Spam filtering is an example of supervised learning.

Supervised learning can be grouped further in two categories of algorithms:

- **Classification**
- **Regression**

2) Unsupervised Learning

Unsupervised learning is a learning method in which a machine learns without any supervision.

The training is provided to the machine with the set of data that has not been labeled, classified, or categorized, and the algorithm needs to act on that data without any supervision. The goal of unsupervised learning is to restructure the input data into new features or a group of objects with similar patterns.

In unsupervised learning, we don't have a predetermined result. The machine tries to find useful insights from the huge amount of data. It can be further classified into two categories of algorithms:

- **Clustering**
 - **Association**
-

3) Reinforcement Learning

Reinforcement learning is a feedback-based learning method, in which a learning agent gets a reward for each right action and gets a penalty for each wrong action. The agent learns automatically with these feedbacks and improves its performance. In reinforcement learning, the agent interacts with the environment and explores it. The goal of an agent is to get the most reward points, and hence, it improves its performance.

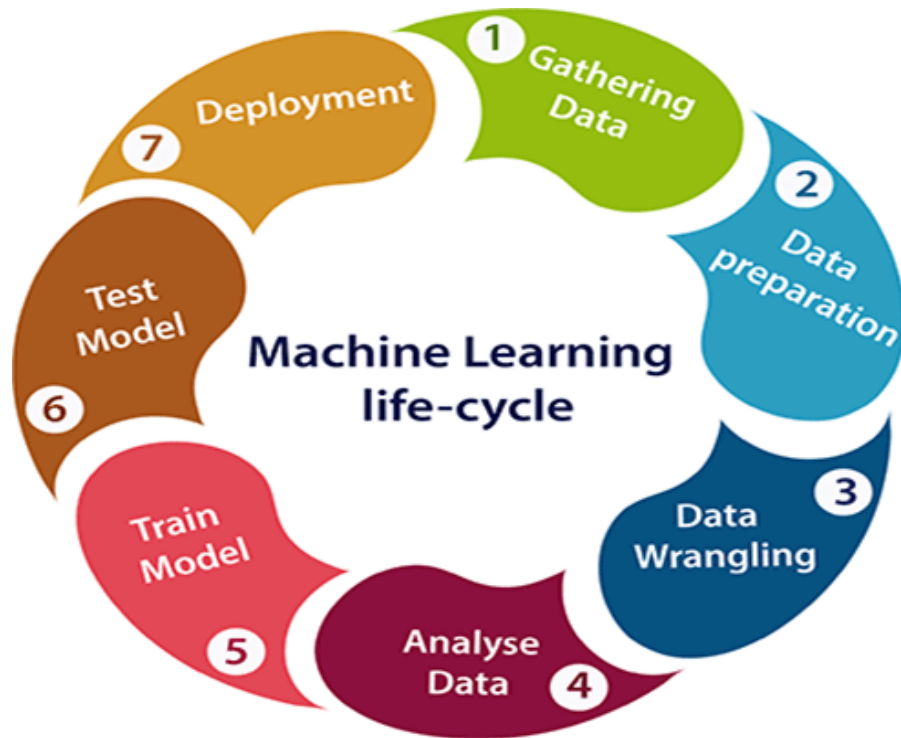
The robotic dog, which automatically learns the movement of his arms, is an example of Reinforcement learning.

Machine learning Life cycle:

Machine learning has given the computer systems the abilities to automatically learn without being explicitly programmed. But how does a machine learning system work? So, it can be described using the life cycle of machine learning. Machine learning life cycle is a cyclic process to build an efficient machine learning project. The main purpose of the life cycle is to find a solution to the problem or project.

Machine learning life cycle involves seven major steps, which are given below:

- **Gathering Data**
 - **Data preparation**
 - **Data Wrangling**
 - **Analyze Data**
 - **Train the model**
 - **Test the model**
 - **Deployment**
-



1. Gathering Data:

Data Gathering is the first step of the machine learning life cycle. The goal of this step is to identify and obtain all data-related problems.

In this step, we need to identify the different data sources, as data can be collected from various sources such as **files**, **database**, **internet**, or **mobile devices**. It is one of the most important steps of the life cycle. The quantity and quality of the collected data will determine the efficiency of the output. The more will be the data, the more accurate will be the prediction.

This step includes the below tasks:

- **Identify various data sources**
- **Collect data**
- **Integrate the data obtained from different sources**

By performing the above task, we get a coherent set of data, also called as a **dataset**. It will be used in further steps.

2. Data preparation:

After collecting the data, we need to prepare it for further steps. Data preparation is a step where we put our data into a suitable place and prepare it to use in our machine learning training.

In this step, first, we put all data together, and then randomize the ordering of data.

This step can be further divided into two processes:

- **Data exploration:** It is used to understand the nature of data that we have to work with. We need to understand the characteristics, format, and quality of data. A better understanding of data leads to an effective outcome. In this, we find Correlations, general trends, and outliers.
 - **Data pre-processing:** Now the next step is preprocessing of data for its analysis.
-

3. Data Wrangling

Data wrangling is the process of cleaning and converting raw data into a useable format. It is the process of cleaning the data, selecting the variable to use, and transforming the data in a proper format to make it more suitable for analysis in the next step. It is one of the most important steps of the complete process. Cleaning of data is required to address the quality issues.

It is not necessary that data we have collected is always of our use as some of the data may not be useful. In real-world applications, collected data may have various issues, including:

- **Missing Values**
 - **Duplicate data**
 - **Invalid data**
 - **Noise**
-

So, we use various filtering techniques to clean the data.

It is mandatory to detect and remove the above issues because it can negatively affect the quality of the outcome.

4. Data Analysis

Now the cleaned and prepared data is passed on to the analysis step. This step involves:

- **Selection of analytical techniques**
 - **Building models**
 - **Review the result**
-

The aim of this step is to build a machine learning model to analyze the data using various analytical techniques and review the outcome. It starts with the determination of the type of the problems, where

we select the machine learning techniques such as **Classification, Regression, Cluster analysis, Association**, etc. then build the model using prepared data, and evaluate the model.

Hence, in this step, we take the data and use machine learning algorithms to build the model.

5. Train Model

Now the next step is to train the model, in this step we train our model to improve its performance for better outcome of the problem.

We use datasets to train the model using various machine learning algorithms. Training a model is required so that it can understand the various patterns, rules, and, features.

6. Test Model

Once our machine learning model has been trained on a given dataset, then we test the model. In this step, we check for the accuracy of our model by providing a test dataset to it.

Testing the model determines the percentage accuracy of the model as per the requirement of project or problem.

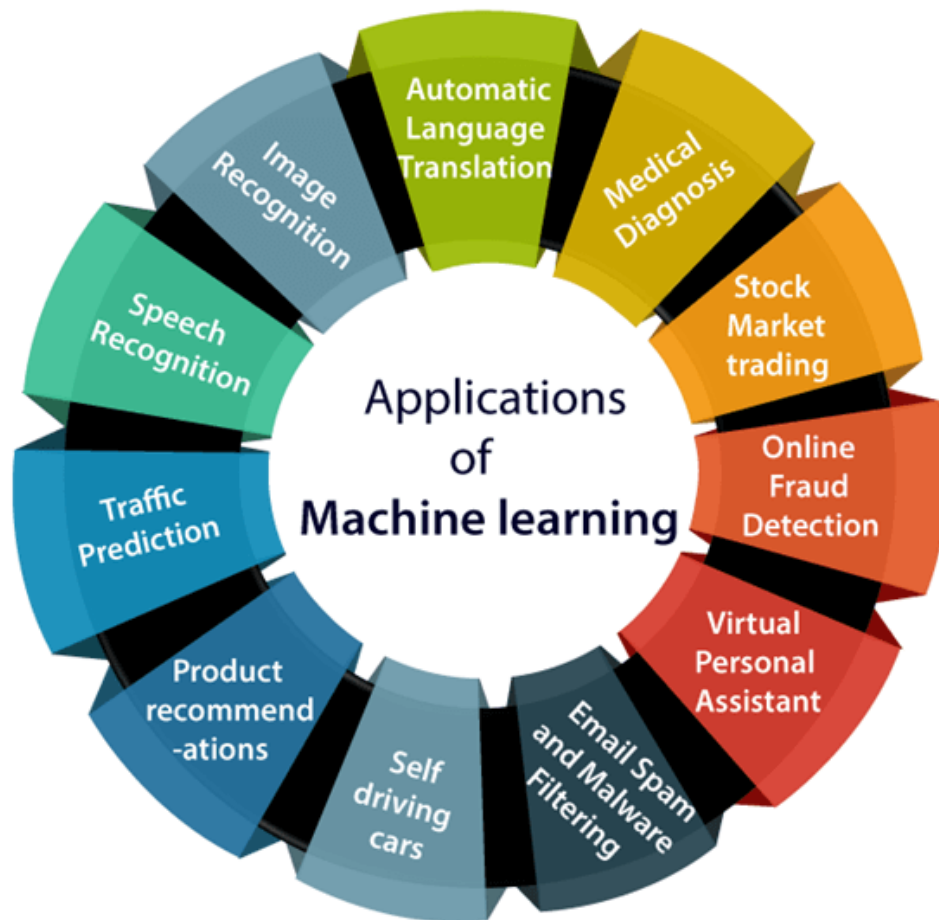
7. Deployment

The last step of machine learning life cycle is deployment, where we deploy the model in the real-world system.

If the above-prepared model is producing an accurate result as per our requirement with acceptable speed, then we deploy the model in the real system. But before deploying the project, we will check whether it is improving its performance using available data or not. The deployment phase is similar to making the final report for a project.

Applications of Machine learning:

Machine learning is a buzzword for today's technology, and it is growing very rapidly day by day. We are using machine learning in our daily life even without knowing it such as Google Maps, Google assistant, Alexa, etc. Below are some most trending real-world applications of Machine Learning:



1. Image Recognition:

Image recognition is one of the most common applications of machine learning. It is used to identify objects, persons, places, digital images, etc. The popular use case of image recognition and face detection is, **Automatic friend tagging suggestion**:

Facebook provides us a feature of auto friend tagging suggestion. Whenever we upload a photo with our Facebook friends, then we automatically get a tagging suggestion with name, and the technology behind this is machine learning's **face detection** and **recognition algorithm**.

It is based on the Facebook project named "**Deep Face**," which is responsible for face recognition and person identification in the picture.

2. Speech Recognition

While using Google, we get an option of "**Search by voice**," it comes under speech recognition, and it's a popular application of machine learning.

Speech recognition is a process of converting voice instructions into text, and it is also known as "**Speech to text**", or "**Computer speech recognition**." At present, machine learning algorithms are widely used by various applications of speech recognition. **Google assistant**, **Siri**, **Cortana**, and **Alexa** are using speech recognition technology to follow the voice instructions.

3. Traffic prediction:

If we want to visit a new place, we take help of Google Maps, which shows us the correct path with the shortest route and predicts the traffic conditions.

It predicts the traffic conditions such as whether traffic is cleared, slow-moving, or heavily congested with the help of two ways:

- **Real Time location** of the vehicle from Google Map app and sensors
- **Average time has taken** on past days at the same time.

Everyone who is using Google Map is helping this app to make it better. It takes information from the user and sends back to its database to improve the performance.

4. Product recommendations:

Machine learning is widely used by various e-commerce and entertainment companies such as **Amazon, Netflix**, etc., for product recommendation to the user. Whenever we search for some product on Amazon, then we started getting an advertisement for the same product while internet surfing on the same browser and this is because of machine learning.

Google understands the user interest using various machine learning algorithms and suggests the product as per customer interest.

As similar, when we use Netflix, we find some recommendations for entertainment series, movies, etc., and this is also done with the help of machine learning.

5. Self-driving cars:

One of the most exciting applications of machine learning is self-driving cars. Machine learning plays a significant role in self-driving cars. Tesla, the most popular car manufacturing company is working on self-driving car. It is using unsupervised learning method to train the car models to detect people and objects while driving.

6. Email Spam and Malware Filtering:

Whenever we receive a new email, it is filtered automatically as important, normal, and spam. We always receive an important mail in our inbox with the important symbol and spam emails in our spam box, and the technology behind this is Machine learning. Below are some spam filters used by Gmail:

- Content Filter
- Header filter
- General blacklists filter
- Rules-based filters
- Permission filters

Some machine learning algorithms such as **Multi-Layer Perceptron, Decision tree**, and **Naïve Bayes classifier** are used for email spam filtering and malware detection.

7. Virtual Personal Assistant:

We have various virtual personal assistants such as **Google assistant, Alexa, Cortana, Siri**. As the name suggests, they help us in finding the information using our voice instruction. These assistants can help us in various ways just by our voice instructions such as Play music, call someone, Open an email, Scheduling an appointment, etc.

These virtual assistants use machine learning algorithms as an important part.

These assistant record our voice instructions, send it over the server on a cloud, and decode it using ML algorithms and act accordingly.

8. Online Fraud Detection:

Machine learning is making our online transaction safe and secure by detecting fraud transaction. Whenever we perform some online transaction, there may be various ways that a fraudulent transaction can take place such as **fake accounts, fake ids, and steal money** in the middle of a transaction. So to detect this, **Feed Forward Neural network** helps us by checking whether it is a genuine transaction or a fraud transaction.

For each genuine transaction, the output is converted into some hash values, and these values become the input for the next round. For each genuine transaction, there is a specific pattern which gets change for the fraud transaction hence, it detects it and makes our online transactions more secure.

9. Stock Market trading:

Machine learning is widely used in stock market trading. In the stock market, there is always a risk of up and downs in shares, so for this machine learning's **long short term memory neural network** is used for the prediction of stock market trends.

10. Medical Diagnosis:

In medical science, machine learning is used for diseases diagnoses. With this, medical technology is growing very fast and able to build 3D models that can predict the exact position of lesions in the brain.

It helps in finding brain tumors and other brain-related diseases easily.

11. Automatic Language Translation:

Nowadays, if we visit a new place and we are not aware of the language then it is not a problem at all, as for this also machine learning helps us by converting the text into our known languages. Google's GNMT (Google Neural Machine Translation) provide this feature, which is a Neural Machine Learning that translates the text into our familiar language, and it called as automatic translation.

The technology behind the automatic translation is a sequence to sequence learning algorithm, which is used with image recognition and translates the text from one language to another language.

UNIT: 2

Dimensionality Reduction

Definition:

While working with machine learning models, we often encounter datasets with a large number of features. These datasets can lead to problems such as increased computation time and overfitting. To address these issues, we use dimensionality reduction techniques.

Dimensionality reduction is the process of reducing the number of features (or dimensions) in a dataset while retaining as much information as possible.

There are several techniques for dimensionality reduction, including [principal component analysis \(PCA\)](#), [singular value decomposition \(SVD\)](#), and [linear discriminant analysis \(LDA\)](#). Each technique uses a different method to project the data onto a lower-dimensional space while preserving important information.

Let's take an example to explain this better:

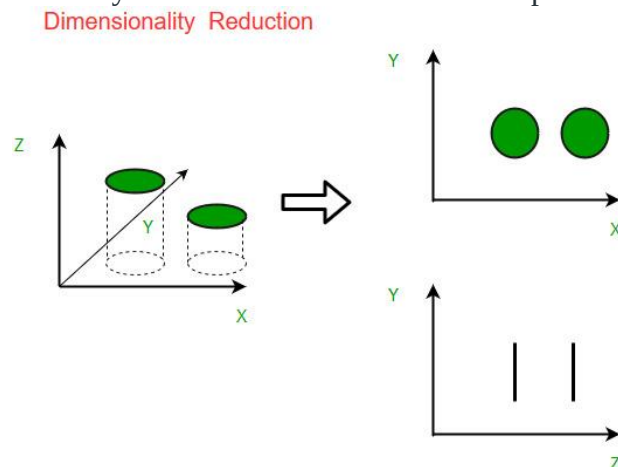
Imagine you are building a machine learning model to predict **house prices** based on features like the number of **bedrooms**, **square footage**, **location**, **age of the house**, **number of bathrooms**, and so on. If you have too many features like additional ones for each room's condition, flooring type, or neighborhood amenities, your dataset can become very large and complex.

Before Dimensionality Reduction:

With too many features, your model may become slow to train, and it might also pick up unnecessary details or noise. For example, if the flooring type doesn't significantly impact house prices, it might lead the model to make less accurate predictions, especially when the data is noisy or when there are many irrelevant features.

How Dimensionality Reduction Works?

Lets understand how dimensionality Reduction is used with the help of the figure below:



On the left, data points exist in a **3D space** (X, Y, Z), but the Z-dimension appears unnecessary since the data primarily varies along the X and Y axes. The goal of dimensionality reduction is to remove less important dimensions without losing valuable information.

On the right, after reducing the dimensionality, the data is represented in **lower-dimensional spaces**. The top plot (X-Y) maintains the meaningful structure, while the bottom plot (Z-Y) shows that the Z-dimension contributed little useful information.

This process makes data analysis more efficient, improving computation speed and visualization while minimizing redundancy.

Advantages of Dimensionality Reduction:

As seen earlier, high dimensionality makes models inefficient. Let's now summarize the key advantages of reducing dimensionality.

- **Faster Computation:** With fewer features, machine learning algorithms can process data more quickly. This results in faster model training and testing, which is particularly useful when working with large datasets.
- **Better Visualization:** As we saw in the earlier figure, reducing dimensions makes it easier to visualize data, revealing hidden patterns.
- **Prevent Overfitting:** With fewer features, models are less likely to memorize the training data and overfit. This helps the model generalize better to new, unseen data, improving its ability to make accurate predictions.

Disadvantages of Dimensionality Reduction:

- **Data Loss & Reduced Accuracy** – Some important information may be lost during dimensionality reduction, potentially affecting model performance.
- **Interpretability Challenges** – The transformed features (e.g., principal components) may not have clear meanings, making it harder to understand relationships in the original data.
- **Choosing the Right Components** – Deciding how many dimensions to keep is difficult, as keeping too few may lose valuable information, while keeping too many can lead to overfitting.

Row vector and column vector:

Row Vector:

A **row vector** is a vector that is represented horizontally. It is a matrix with one row and multiple columns. For example:

(a b c)

Here, a , b , and c are the components of the vector.

Column Vector:

A **column vector**, on the other hand, is a vector that is represented vertically. It is a matrix with one column and multiple rows. For example:

(a
b
c)

Again, a , b , and c are the components of the vector.

Key Differences

- **Orientation:** The most obvious difference is the orientation—row vectors are horizontal, while column vectors are vertical.
- **Matrix Multiplication:** When performing matrix multiplication, the orientation of vectors matters. A row vector can be multiplied by a matrix on its right, while a column vector can be multiplied by a matrix on its left.
- **Usage in Operations:** In linear algebra, column vectors are more commonly used for representing points in space or as inputs to linear transformations. Row vectors are often used to represent linear functionals or as outputs of certain operations.

Example of Matrix Multiplication:

Consider a matrix A and a column vector v :

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 \end{pmatrix}, v = \begin{pmatrix} x \\ y \end{pmatrix}$$

The product Av is defined and results in another column vector:

$$Av = \begin{pmatrix} 1x + 2y \\ 3x + 4y \end{pmatrix}$$

However, if v were a row vector, the product vA would be defined differently and would result in a row vector.

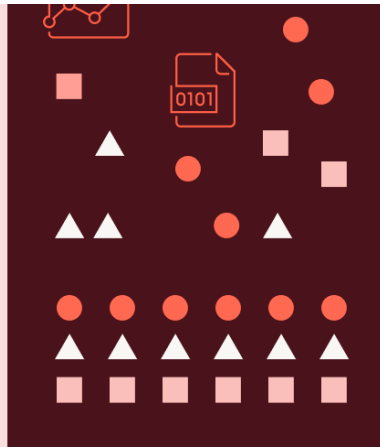
What is Dataset?

A **Dataset** is a set of data grouped into a collection with which developers can work to meet their goals. In a dataset, the rows represent the number of data points and the columns represent the features of the Dataset. They are mostly used in fields like machine learning, business, and government to gain insights, make informed decisions, or train algorithms. Datasets may vary in size and complexity and they mostly require cleaning and preprocessing to ensure data quality and suitability for analysis or modeling.

A **dataset** is a collection of data, typically organized in a structured format, used for analysis, reporting, or machine learning model training. Datasets can contain various types of data, such as numbers, text, images, audio files, or other forms of information. They are often used in fields like data science, artificial intelligence, business intelligence, and scientific research to support decision-making, predict outcomes, or understand patterns and trends.

What is a Dataset?

A dataset is a collection of data that can be used for analytics or to train machine learning models



Key Characteristics of a Dataset:

1. **Structure:** Datasets are usually structured, meaning they are organized into rows and columns (like a spreadsheet) or into other formats that allow for easy access and manipulation.
2. **Variety:** Datasets can include different types of data, such as numerical, categorical, text, or multimedia.
3. **Size:** Datasets can range from small (a few records) to extremely large (big data), depending on the application.
4. **Purpose:** Datasets are often created or curated for specific purposes, such as training machine learning models, conducting statistical analysis, or supporting business decisions.

Types of Dataset:

- **Numerical Dataset:** They include numerical data points that can be solved with equations. These include temperature, humidity, marks and so on.
- **Categorical Dataset:** These include categories such as colour, gender, occupation, games, sports and so on.
- **Ordered Dataset:** These datasets contain data that are ordered in ranks, for example, customer reviews, movie ratings and so on.

Examples of Datasets:

- **Customer Purchase History:** A dataset containing customer IDs, purchase dates, items bought, and amounts spent.
- **Weather Data:** A dataset with temperature, humidity, wind speed, and other meteorological variables over time.
- **Image Classification Dataset:** A collection of images labeled with their respective categories (e.g., animals, vehicles).

How to represent a dataset?

Representing a dataset involves organizing and structuring the data in a way that makes it easy to understand, analyze, and visualize. Here are some common methods to represent a dataset:

1. Tabular Representation:

Data is often represented in a tabular format, where each row represents a single observation or record, and each column represents a variable or feature. This is commonly seen in spreadsheets or data frames in programming languages like Python (using Pandas) or R.

Example:

ID	Name	Age	Gender
1	John	25	Male
2	Jane	30	Female
3	Joe	28	Male

2. Matrix Representation:

In linear algebra and machine learning contexts, datasets can be represented as matrices. Each row of the matrix corresponds to a data point, and each column corresponds to a feature.

Example:

For a dataset with features Age and Income:

```
(25  50000
 30  60000
 28  55000)
```

3. Graphical Representation:

Visualizations such as plots, charts, and graphs are powerful tools for representing datasets. Common types include:

- **Bar Charts:** Useful for categorical data.
- **Scatter Plots:** Show relationships between two continuous variables.
- **Histograms:** Display distributions of a single variable.
- **Heatmaps:** Often used for showing correlations between variables.

4. Statistical Summary:

Representing a dataset through statistical summaries provides an overview of its characteristics. This includes measures like:

- **Mean:** Average value of a variable.
- **Median:** Middle value when data is sorted.
- **Standard Deviation:** Measures variability.
- **Correlation Coefficients:** Show relationships between variables.

5. Data Dictionaries:

For complex datasets, a data dictionary can be used to document each variable, including its name, data type, and description. This helps in understanding the meaning and context of the data.

Tools for Representation:

- **Spreadsheets (e.g., Excel, Google Sheets):** Useful for tabular data.
- **Data Visualization Libraries (e.g., Matplotlib, Seaborn in Python; ggplot2 in R):** For creating plots and charts.
- **Statistical Software (e.g., R, Python libraries like Pandas and NumPy):** For statistical summaries and matrix operations.

Choosing the right method depends on the nature of the data, the purpose of the analysis, and the audience for the representation.

How to represent a dataset as a matrix:

In **machine learning**, the **data** often **comes in multi-dimensional arrays** so **matrices** are **best to handle** such inputs. So we need to have some general idea on basic arithmetic operations in matrices. In this article, we will discuss about matrices and matrix arithmetic for machine learning.

What is a Matrix?

Matrix is a two dimensional array where the **data** is in the **form of rows and columns**. The each element in the matrix is **represented** as **$a[i][j]$** where **i represents the row number and j represents the column number**. Each matrix has an order representing the total number of rows (m) and columns (n) in the matrix and it is written as **m X n**.

Defining a matrix:

In Python, we use **numpy module to define a matrix**. As we discussed that matrices are two dimensional arrays, let us create a two dimensional array (matrix) using **numpy.array() method**.

```
#importing numpy module  
import numpy as np
```

```
#defining a matrix using np.array() method
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
```

```
#printing the matrix
print(matrix)
```

Output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

we can see that it is in the form of list of lists which is a two-dimensional array having 3 rows and 3 columns. The order of the above matrix is 3 X 3.

Matrix Arithmetic:

Matrix Addition:

Addition of matrices can only be done **when the two matrices have the same order**. It is done by **using (+) operator** which adds corresponding elements of the matrices. For example, if we have two matrices A and B then the $a[i][j]$ is added with $b[i][j]$ where i is row number and j is column number.

```
import numpy as np
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
```

```
#adding A and B matrices
```

```
C = A + B
```

```
print(C)
```

Output:

```
[[ 6  8]
 [10 12]]
```

Explanation : In the above example, we have two matrices A and B.

$A = \begin{bmatrix} 1 & 2 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \end{bmatrix}$

$\begin{bmatrix} 3 & 4 \end{bmatrix}$ $\begin{bmatrix} 7 & 8 \end{bmatrix}$

$C = \begin{bmatrix} 1+5 & 2+6 \end{bmatrix}$ $C = \begin{bmatrix} 6 & 8 \end{bmatrix}$

$\begin{bmatrix} 3+7 & 4+8 \end{bmatrix} ==> \begin{bmatrix} 10 & 12 \end{bmatrix}$

We can see that the corresponding elements are added and we get the final matrix with the same order.

Matrix Subtraction:

Subtraction of matrices is possible when both the **matrices have same order**. Subtraction is done by **using (-) operator** in Python. When we perform subtraction on two matrices then the elements from the first matrix are subtracted with their corresponding elements in second matrix.

```
import numpy as np
A = np.array([[1 , 2],[3 , 4]])
```

```
B = np.array([[0 , 1],[2 , 1]])
C = A - B
print(C)
```

Output:

```
[[1 1]
 [1 3]]
```

Explanation: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}$
 $C = \begin{bmatrix} 1-0 & 2-1 \\ 3-2 & 4-1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 3 \end{bmatrix}$

The elements of the matrix A are subtracted with elements of matrix B and we got the resultant matrix with order 2 X 2.

Matrix Division:

Division of [matrices](#) is done by **using (/) operator**. It divides elements from first matrix with corresponding elements in the second matrix. The matrices should have same dimensions.

```
import numpy as np
A = np.array([[4 , 2],[6 , 8]])
B = np.array([[2 , 2],[3 , 4]])
C = A/B
print(C)
```

Output:

```
[[2 1]
 [2 2]]
```

Explanation : $A = \begin{bmatrix} 4 & 2 \\ 6 & 8 \end{bmatrix}$ $B = \begin{bmatrix} 2 & 2 \\ 3 & 4 \end{bmatrix}$
 $C = \begin{bmatrix} 4/2 & 2/2 \\ 6/3 & 8/4 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 2 & 2 \end{bmatrix}$

Matrix - Matrix Multiplication (Dot product):

Multiplication of [matrices](#) can only be done when the columns of first matrix is equal to the rows of the second matrix. Suppose there is a matrix **A** of order **m X n** and matrix **B** of order **n X k** so here **columns of matrix A = rows of matrix B = n** and the order of the resulting matrix $C = AB$ is **m X k**. Hence multiplication is possible for the above matrices.

Note : Matrices do not satisfy commutative property

AB is not equal to BA

```
import numpy as np
A = np.array([[1, 2],[3, 4]])
B = np.array([[5, 6],[7, 8]])
C = A.dot(B)
print(C)
```

Output:

```
[[19 22]
 [43 50]]
```

Explanation : $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1*5 + 2*7 & 1*6 + 2*8 \\ 3*5 + 4*7 & 3*6 + 4*8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

$[3, 4] \times [7, 8] \Rightarrow [3 \times 5 + 4 \times 7 \quad 3 \times 6 + 4 \times 8] \Rightarrow [43 \ 50]$

The matrix A is of order 2 X 2 and matrix B of order 2 X 2. Since the columns of A = rows of B = 2, the multiplication is possible. The multiplication is done in a way where the 1st row of A is multiplied with 1st and 2nd column of B giving us $C[0][0]$ and $C[0][1]$ and the 2nd row of A is multiplied with 1st and 2nd column of B giving us $C[1][0]$ and $C[1][1]$.

Matrix - Vector multiplication:

A **vector** is a one dimensional array having **either a single row or single column**. A matrix having **only one row** is called as a **row vector** and matrix having **only one column** is called as a **column vector**. While multiplying the matrix with a vector, we need to check the multiplication condition which is the columns of first matrix = rows of the second matrix.

```
import numpy as np
A = np.array([[1, 2], [1, 1]])
V = np.array([[1], [1]])

#matrix - vector multiplication
C = A.dot(V)
print(C)
```

Output:

```
[[3]
 [2]]
```

Explanation : We multiplied the matrix A with vector V and we got a column vector C. In the same way we can multiply a matrix with a row vector and the dot product will also be a row vector.

$A = [1, 2] \quad V = [1] \quad [1 \times 1 + 2 \times 1] \quad [3]$
 $[1, 1] \quad [1] \Rightarrow A \cdot V = [1 \times 1 + 1 \times 1] \Rightarrow A \cdot V = [2]$

Matrix - Scalar multiplication:

When we **multiply a matrix with a scalar** then it is **multiplied with each and every element** in the matrix. The **order remains same** even after multiplying the matrix with scalar. We **use matrix - scalar multiplications** while **solving equations in linear algebra**.

```
import numpy as np
A = np.array([[1,2],[3,4]])
b = 2
print(A * b)
```

Output:

```
[[2 4]
 [6 8]]
```

Explanation : In the above matrix, we can see that $b=2$ is multiplied with each element in the matrix.

$A \times b = [1 \times 2, 2 \times 2] \quad [2, 4]$
 $[3 \times 2, 4 \times 2] \Rightarrow [6, 8]$

Data Preprocessing in Machine Learning:

Normalization:

Normalization is one of the most frequently used data preparation techniques, which helps us to change the values of numeric columns in the dataset to use a common scale.

- "Normalization" alone can refer to a broader concept of adjusting values into a standard range.

It is not necessary for all datasets in a model. It is required only when features of machine learning models have different ranges.

Mathematically, we can calculate normalization with the below formula:

$$X_n = (X - X_{\text{minimum}}) / (X_{\text{maximum}} - X_{\text{minimum}})$$

- X_n = Value of Normalization
- X_{maximum} = Maximum value of a feature
- X_{minimum} = Minimum value of a feature

Example: Let's assume we have a model dataset having maximum and minimum values of feature as mentioned above. To normalize the machine learning model, values are shifted and rescaled so their range can vary between 0 and 1. This technique is also known as **Min-Max scaling**. In this scaling technique, we will change the feature values as follows:

Case1- If the value of X is minimum, the value of Numerator will be 0; hence Normalization will also be 0.

$$X_n = (X - X_{\text{minimum}}) / (X_{\text{maximum}} - X_{\text{minimum}})$$

Put $X = X_{\text{minimum}}$ in above formula, we get;

$$X_n = X_{\text{minimum}} - X_{\text{minimum}} / (X_{\text{maximum}} - X_{\text{minimum}})$$

$$X_n = 0$$

Case2- If the value of X is maximum, then the value of the numerator is equal to the denominator; hence Normalization will be 1.

$$X_n = (X - X_{\text{minimum}}) / (X_{\text{maximum}} - X_{\text{minimum}})$$

Put $X = X_{\text{maximum}}$ in above formula, we get;

$$X_n = X_{\text{maximum}} - X_{\text{minimum}} / (X_{\text{maximum}} - X_{\text{minimum}})$$

$$X_n = 1$$

Case3- On the other hand, if the value of X is neither maximum nor minimum, then values of normalization will also be between 0 and 1.

Hence, Normalization can be defined as a scaling method where values are shifted and rescaled to maintain their ranges between 0 and 1, or in other words; it can be referred to as **Min-Max scaling technique**.

Feature Normalization in Machine Learning:

Feature normalization is a preprocessing technique used in machine learning to standardize the range or distribution of input features. It ensures that all features contribute equally to the learning process and prevents certain features from dominating due to differences in scale.

- This term specifically refers to the process of normalizing the features (input variables) of a dataset in machine learning.
- The goal is to scale the features so they have similar ranges, preventing any single feature from dominating the learning process.

Why is Feature Normalization Important?

1. **Improves Model Convergence** – Many machine learning algorithms, such as gradient descent, converge faster when features are on a similar scale.
2. **Prevents Dominance of Large-Scale Features** – Features with large ranges can overshadow smaller features, leading to biased models.
3. **Enhances Stability and Performance** – Normalized features improve numerical stability, especially in distance-based models (e.g., KNN, SVM, clustering).
4. **Required for Certain Algorithms** – Some algorithms like Principal Component Analysis (PCA) assume that data is centered and scaled.

Common Feature Normalization Techniques

1. Min-Max Normalization (Rescaling)

- Scales features to a fixed range, usually [0,1][0,1][0,1] or [-1,1][-1,1][-1,1].
- Formula:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- **Example:** If a feature has values between 10 and 100, Min-Max scaling would transform 10 to 0 and 100 to 1.
- **Use Case:** When data distribution is not Gaussian, and you need bounded values.

2. Z-score Normalization (Standardization)

- Transforms data to have a mean of 0 and a standard deviation of 1.
- Formula:

$$X' = (X - \mu) / \sigma$$

$$X' = \frac{X - \mu}{\sigma}$$

Where μ is the mean and σ is the standard deviation.

- **Example:** If a feature has a mean of 50 and a standard deviation of 10, a value of 60 becomes $\frac{60 - 50}{10} = 1.0$.
- **Use Case:** When features have different units or follow a normal distribution.

3. Mean Normalization

- Centers data around 0 but scales within a range.
- Formula:

$$X' = \frac{X - \mu}{X_{\max} - X_{\min}}$$

$$X' = \frac{X - \mu}{X_{\max} - X_{\min}}$$

- **Use Case:** Similar to standardization but ensures values are within a specific range.

4. Unit Vector Normalization (L2 Normalization)

- Ensures the magnitude of feature vectors is 1.
- Formula:

$$X' = X / ||X||$$

$$X' = \frac{X}{||X||}$$

Where $||X||$ is the Euclidean norm.

- **Use Case:** Used in text classification (TF-IDF) and deep learning to normalize input vectors.
-

Which Normalization Method Should You Use?

- **Min-Max Scaling** → When you need bounded values (e.g., image pixel values).
- **Z-score Normalization** → When features have different units or a normal distribution is assumed.
- **L2 Normalization** → When working with vectors and similarity-based models (e.g., NLP, deep learning).

* Let's understand it by an example:

- Imagine that you are training a Machine Learning model that analyzes how the height and weight of a person affect health.
- Suppose that the **height data is measured in meters (m)** and the **weight data is measured in kilograms (kg)**, most of the height data will be within the range of **1.6–1.8m** while most of the weight data will be within the range of **50–100 kg**.

	Weight	Height
Data entry 1	58kg	1.6m
Data entry 2	70kg	1.8m
Data entry 3	45kg	1.53m

What is the issue?

Well, one important concept to understand is that a computer does not understand any context, whether this number has a meaning. This would mean that, from the computer's point of view, they are just a bunch of numbers.

When a computer looks at these data, it will think, the **height data** has such a small scale and the **weight data** has a much larger scale, which means the weight data must be more important, let's put the focus on the weight data more!

By performing normalization, the model will train faster, and the parameters update will be more equal.

Now, you see the problem. Both the height and the weight data are equally important to analyze the problem.

So, how do we overcome this? Easy, simply rescaling it.

Talking about rescaling, there are two major ways of performing normalization.

Min-Max Scaling

$$X_{\text{normalized}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Formula of Min-Max Scaling

The above formula is simple, don't you worry.

- **X(normalized):** The rescaled value of your data feature.
- **X(min):** The minimum value of your entire data feature.
- **X (max):** The maximum value of your entire feature.

- **X:** Each of the data entry to be rescaled.

Min-Max scaling rescales the original data into values between **0 and 1**. It maintains the ratio of the original feature.

Z-Score Normalization (Standardization)

This technique rescales the original feature according to the statistic value. In other words, it transforms the data to make the entire data feature to have the **average value of 0**, and the **standard deviation of 1**.

$$z = \frac{x - \mu}{\sigma}$$

Formula of Z-Score Normalization

This formula is straight forward too, don't you worry.

- **z:** The rescaled value.
- **x:** Each of the data entry to be rescaled.
- **μ:** The average of the entire data feature.
- **σ:** The standard deviation of entire data feature.

As mentioned above, this technique does not rescale the data into a definite range, but instead, it adjusts the entire data feature to **average of 0** and **standard deviation of 1**.

Which to use?

The answer is pretty simple too.

Rule of thumb:

- When your features are normally distributed, min-max normalization tends to work well.
- If not, apply z-score normalization.
- In fact, z-score normalization works well most of the time.

There you have it! In the real-world application, most of the numerical features require normalization, no matter what. Only certain algorithm does not require normalization, such as decision tree as it operates on Information Gain Ratio.

Prerequisite for Principal Component Analysis (PCA) :

Principal Component Analysis (PCA) is nothing but the dimension reduction technique used to reduce the dimensions by forming linear combinations of features which retain maximum information/variance of data.

This is slightly confusing right now for total beginners to understand since concept like retaining information (variance) is mentioned above. Don't worry, if you do not get it at this moment we will revisit it once again once you learn all prerequisites.

However, if in doubt, just perform normalization, as it does not hurt anyway!

We will visit the below-mentioned concepts that required to understand PCA thoroughly —

1. Vector (Row & Column)
2. How to represent dataset as a matrix?
3. Mean
4. Variance

5. Standardization of dataset
6. Covariance & Covariance Matrix
7. Eigen Value & Eigen Vectors

Let's visit them one-by-one.

1. Row Vector & Column Vector:

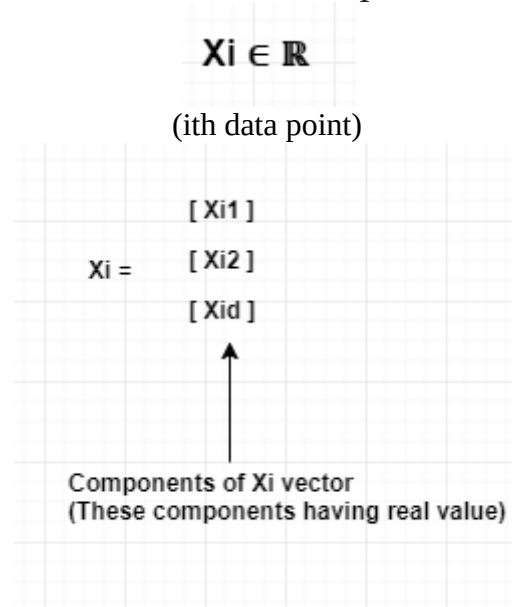
In linear algebra the data points that are real-valued represented as

$$\mathbf{X_i} \in \mathbb{R}$$

(ith data point)

$$\mathbf{X_i} = \begin{bmatrix} X_{i1} \\ X_{i2} \\ \vdots \\ X_{id} \end{bmatrix}$$

Components of $\mathbf{X_i}$ vector
(These components having real value)



Each column is considered a vector is known as a column vector. In the above figure, it is clear that $\mathbf{X_i}$ is the column vector. (No information is available by default is column vector)

The transpose of a column vector is known as a row vector. Each row in a dataset is considered as a row vector. In the above example, if you take the transpose then we will get a row vector.

Example:

column vector

144
13
25
108
96
61
73
60
48
109

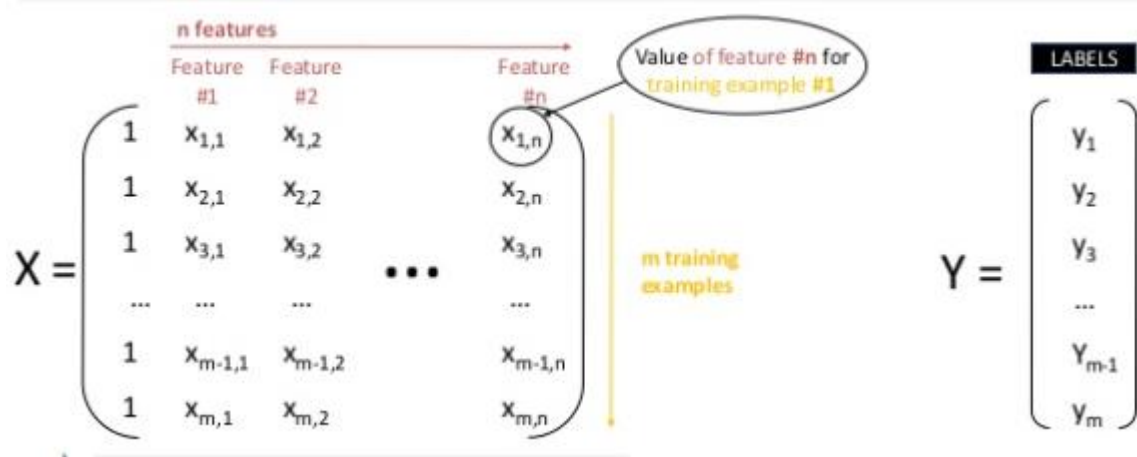
row vector

3	141	140	6	7	137	136	10	11	133
---	-----	-----	---	---	-----	-----	----	----	-----

A column vector is on the left side, and a row vector is shown on the right side.

2. How to represent dataset as a matrix:

Each row in the dataset is considered as a data point and each column in the dataset is considered as features.



3. Mean:

The mean is the average of the numbers. It is very easy to calculate: add up all the numbers, then divide how many numbers there are.

$$\frac{M + E + A + N}{4}$$

4. Variance & Standard Deviation:

The variance is defined as-

“The average of the squared differences from the mean”

The variance tells us how much your data has spread out from the mean of the dataset.

Sample Variance (s^2)

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$$

s^2 = variance

x_i = term in data set

$\bar{x}$ = Sample mean

$\sum$ = Sum

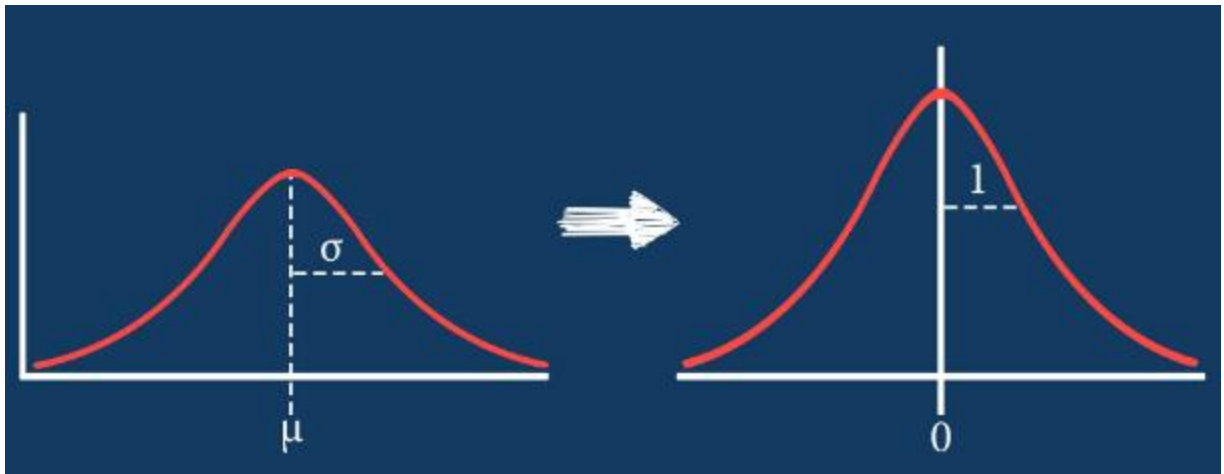
n = Sample size

wikiHow to Calculate Variance

Once we have variance we can easily calculate a standard deviation once we have variance. Standard deviation is nothing but the square root of the variance.

5. Standardization of Dataset:

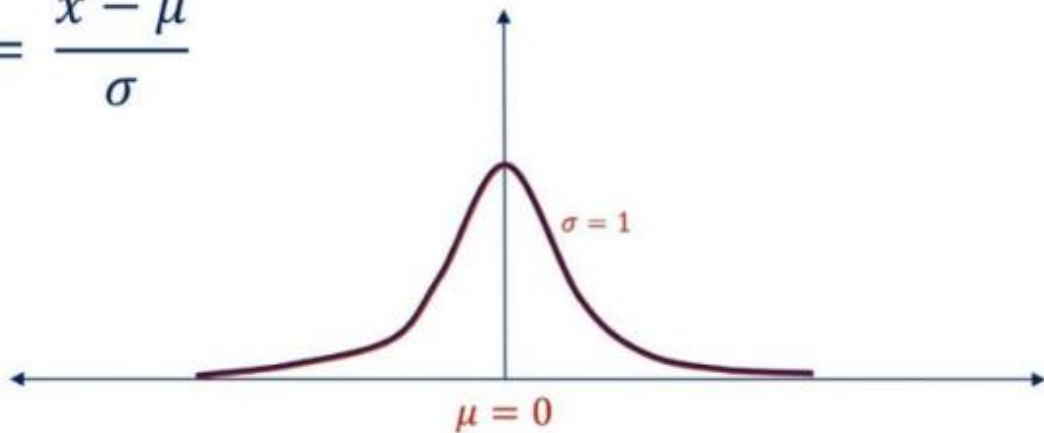
Sometimes many features come with different scales. It means these variables or feature do not give an equal contribution to the analysis. Let's say you've two variables height(cm) of the student and weight (KG). Since both the variable are having different scales cause a serious issue in our analysis. When we standardize variable we make sure that the mean of the feature is 0 and the standard deviation is equal to 1. By doing this we can get rid of scales.



How do we standardize our data?

Z-score standardization is one of the most popular methods to normalize data. In this case, we rescale an original variable to have a **mean of zero** and a **standard deviation of one**.

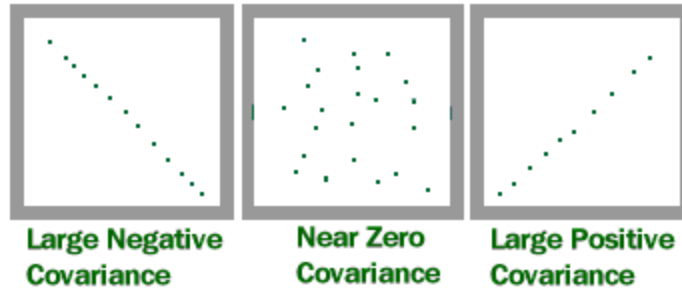
$$Z = \frac{x - \mu}{\sigma}$$



6. Covariance & Covariance Matrix:

Covariance is one of a family statistical measures used to analyse the linear relationship between two variables. **How do two variables behave as a pair?** Covariance can take any value between **-Inf to +Inf**. A positive value indicates an increasing linear relationship and a negative value indicates a decreasing relationship. Covariance near to **ZERO** indicates no relationship between the variables.

COVARIANCE



$$\text{COV} = \frac{\sum_{i=1}^n (X_i - \bar{x})(Y_i - \bar{y})}{n - 1}$$

How to calculate COV(X, Y)

Note: Covariance does not tell you about ‘Strength of relationship’. It just tells you whether there is a positive or negative linear relationship exists or not.

These all the information that you require to understand Covariance. Now let’s move towards Covariance Matrix and see what is it?

A covariance matrix is nothing but the information about auto-covariance (Covariance with itself) and Covariance between two different variables presented in matrix format as shown below-

$$C = \begin{bmatrix} \text{cov}(X, X) & \text{cov}(X, Y) & \text{cov}(X, Z) \\ \text{cov}(Y, X) & \text{cov}(Y, Y) & \text{cov}(Y, Z) \\ \text{cov}(Z, X) & \text{cov}(Z, Y) & \text{cov}(Z, Z) \end{bmatrix}$$

Covariance Matrix (dimensions = 3)

If you observe the above matrix carefully you can see all diagonal elements are nothing but the covariance with itself. These are also called variance of the variable. Thus we can say,

$$\text{Cov}(X, X) = \text{Var}(X) \mid \text{Cov}(Y, Y) = \text{Var}(Y) \mid \text{Cov}(Z, Z) = \text{Var}(Z)$$

X

Let's calculate the covariance of two columns $\text{COV}(F_1, F_2)$ to understand the concept that we discussed above in a better way.

$$Cov(F1, F2) = \sum_{i=1}^n \frac{(X_{i1} - \mu_1) * (X_{i2} - \mu_2)}{n}$$

Covariance Between 2 Features

where,

X_{i1} : Data points of F1 | X_{i2} : Data points of F2 | μ_1 & μ_2 : Mean of the respective columns.

Lets assume data is standardized which means $\mu = \mathbf{0}$ & $\sigma = \mathbf{1}$. Therefore the above equation becomes-

$$Cov(F1, F2) = \sum_{i=1}^n X_{i1} * X_{i2}$$

Covariance when $\mu = 0$ & $\sigma = 1$

Therefore,

$$Cov(F1, F2) = \sum_{i=1}^n F_1^T * F_2$$

Thus we can generalize it like below

$$S = X^T X$$

Covariance Matrix

where S = Covariance Matrix | X = Datapoints

Important Points : →

- In tasks like **Principal Component Analysis (PCA)**, the exact values of the covariance matrix are less important than the **directions of variance** (eigenvectors). The scaling factor $1/n$ does not change the directions (eigenvectors), it only scales the eigenvalues (the magnitude of variance).
- So, for PCA, XTX is sufficient to find the principal components, as the scaling factor won't affect the results of eigenvector computations.
- Dividing by n (the number of data points) adds a step that might not be strictly necessary: When dealing with large datasets, avoiding unnecessary operations can save computation time. And Matrix operations like XTX are already computationally intensive, so omitting $1/n$ simplifies the process.

7. Eigen Value & Eigen Vector

The word 'Eigen' is taken from 'German' which means 'Self or own'. It describes a special relationship between two things. For example, "mein eigenes Auto" in German means "My Own Car". Here eigen is explaining the relationship between 'car and my own'. Thus we can think of the same relationship of eigenvectors with a square matrix. In the world of linear algebra, all the square matrix (for example, 2×2 or 4×4) have eigenvectors associated with them.

Let A be the square matrix of order $(n \times n)$, then a number (Real or Complex) λ is said to be the eigenvalue of matrix A if there exists a column matrix x of order $(n \times 1)$ such that -

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

The diagram shows the equation $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$. A callout box labeled "Eigenvector of Matrix A" points to the variable $\mathbf{x}$. Another callout box labeled "Eigenvalue of Matrix A" points to the variable λ .

Therefore,

$$\mathbf{A}\mathbf{x} - \lambda\mathbf{x} = \mathbf{0}$$

$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0}$$

where $\mathbf{I}$ = Identity matrix.

In order to find eigenvalues and eigenvectors first, we need to convert the above equation to a characteristic equation -

$$|\mathbf{A} - \lambda\mathbf{I}| = 0 \dots (\text{Characteristic Equation})$$

Here basically we are subtracting λ value from the diagonal of the square matrix $\mathbf{A}$ then we are taking the determinant of the resulting matrix. Thus the solution to the characteristic equation will give you eigenvalues.

Note: Total number of eigenvalues is same as an order of square matrix. Let say we have 2 x 2 data matrix then there will be 2 eigenvalues. And we find eigenvectors corresponding to eigenvalues.

Once we get λ value by using the above equation then put the value of λ in the equation $(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0}$ to get the corresponding eigenvector.

Confused? Let me Simplify?

In the world of mathematics, specifically in an area called linear algebra, there are these things called matrices and vectors. Imagine a matrix as a big spreadsheet where you store numbers in rows and columns. A vector is like a single column or a single row of this spreadsheet — it's just a list of numbers.

What are Eigenvalues and Eigenvectors?

Let's break down these fancy terms:

1. **Eigenvector:** It's like a magic arrow (vector) that, when a transformation (described by a matrix) is applied to it, it may stretch or shrink but doesn't change its direction. If you have a shape and you squish or stretch it, most directions change — but the eigenvector's direction stays the same. Most other vectors (that are not eigenvectors) will change their direction when the same transformation is applied.
2. **Eigenvalue:** This tells us how much the eigenvector gets stretched or squished during the transformation. If the eigenvalue is 3, the eigenvector gets three times longer. If it's $1/2$, the eigenvector gets squished to half its length.

Why use Determinants?

Okay, this is where it gets a bit tricky but stay with me. A determinant is a special number that we can calculate from a matrix. It's a bit like taking the temperature or the pulse of the matrix to see if it's "healthy" or not.

In the world of matrices:

- If the determinant is zero, the matrix is considered "sick" (in mathematical terms, singular or not invertible), meaning it squashes everything into a flatter space.

- If the determinant isn't zero, the matrix is "healthy" (non-singular or invertible), and it doesn't completely flatten everything.

When we are searching for eigenvalues, we look exactly for those "sick" moments. Why? Because at those moments, our special eigenvectors (remember, those magic arrows) show their true power: they help us find the directions that do not change, even when everything else might be getting squished or stretched.

How does it all come together?

Here's the recipe:

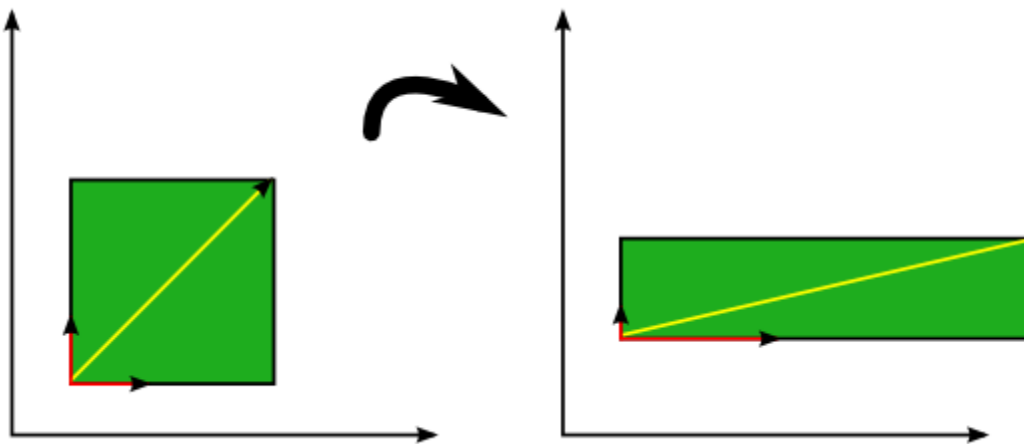
1. **Start with a matrix:** This represents a transformation, like squishing, stretching, or rotating space.
2. **Find the eigenvalues:** These are special numbers where, if we subtract them from our matrix in a specific way and then check our matrix's "health" (by taking its determinant), we find it's "sick." This special operation is like saying, "What value makes this transformation special?"
3. **Find the eigenvectors:** For each special number (eigenvalue), there's a special direction (eigenvector) that keeps its own identity under the transformation. We find it by solving a special equation.
4. **Understand the transformation:** The eigenvalues and eigenvectors tell us about the fundamental nature of the transformation the matrix represents. They tell us which directions stay unchanged and how much other directions get stretched or squished.

In summary, in the world of linear transformations, eigenvalues and eigenvectors help us understand the essence of what a transformation does, by pointing out which directions in space remain constant and how much they change. The determinant helps us find those special

values by identifying the points at which the matrix behaves in a “singular” way, giving us a window into the structure and effects of linear transformations.

Important Pointers:-

1. If you multiply matrix A with vector X then we will get another vector where we can say matrix performed linear transformation on an input vector
2. An eigenvector is a vector whose direction remains unchanged when a linear transformation is applied to it.



Eigenvectors (red) do not change direction when a linear transformation (e.g. scaling) is applied to them. Other vectors (yellow) do.

The green square is only drawn to illustrate the linear transformation that is applied to each of these three vectors.

Let's understand above with mathematical example -

Example:

Let's take a simple matrix A :

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

Matrix A

Case 1: General Vector (Not an Eigenvector)

$$\text{Take } v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Vector v

Multiply A·v

$$A \cdot v = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Multiplication of two vector A·v

The transformed vector [2 3] points in a **different direction** than the original vector [1 1]. So v is **not an eigenvector**.

Case 2: Eigenvector

$$w = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Vector w

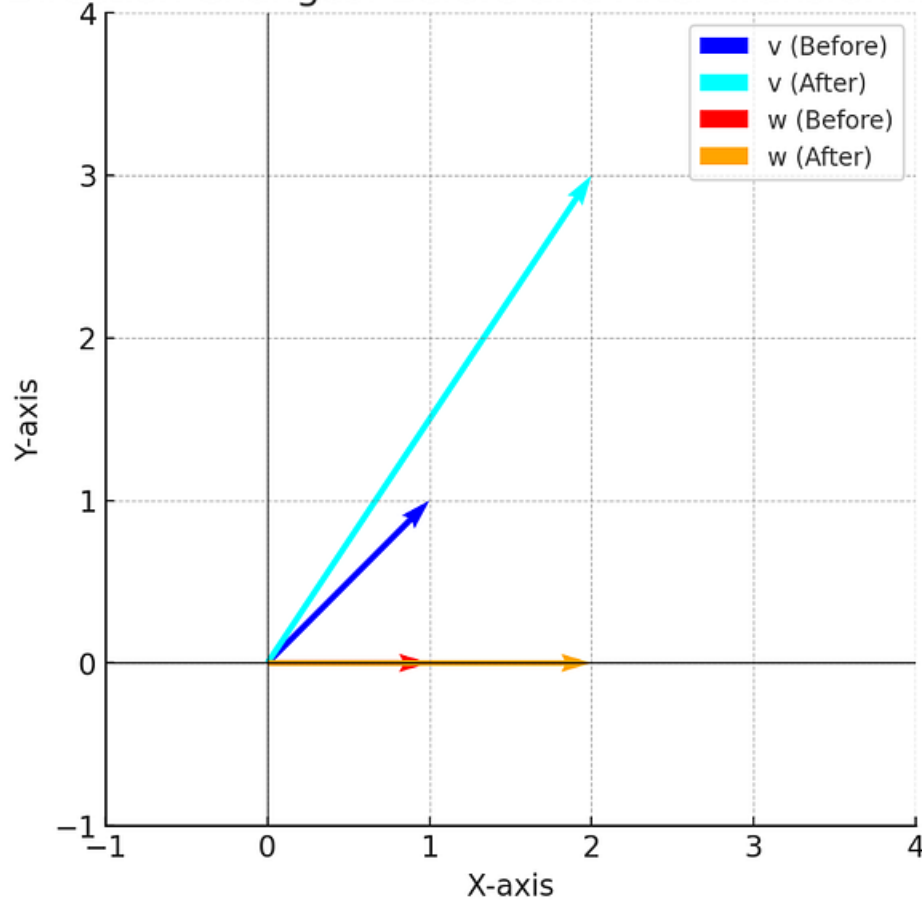
Multiply A·w:

$$A \cdot w = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Multiplication of two vector A·w

The transformed vector [2 0] points in the **same direction** as [1 0]. So w is an **eigenvector** with eigenvalue $\lambda=2$.

Direction Changes: Before and After Transformation



3. Here square matrix is nothing but our calculated covariance matrix and we are using that to find eigenvalues and eigenvectors. Let $\mathbf{S}$ be the covariance matrix then corresponding eigenvalue and eigenvectors are -

$$\text{Eigenvalues}(\mathbf{S}) = \lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_d$$

$$\text{Eigenvectors}(\mathbf{S}) = \mathbf{V}_1 \geq \mathbf{V}_2 \geq \mathbf{V}_3 \geq \dots \geq \mathbf{V}_d$$

Where λ_1 = The largest eigenvalue represents the direction with the **maximum variance** & $\mathbf{V}_1$ is an eigenvector corresponding to maximal eigenvalues

Thus we can say,

$$\mathbf{V_i} \perp \mathbf{V_j} = \mathbf{V_i} * \mathbf{V_j} = ||\mathbf{V_i}|| * ||\mathbf{V_j}|| * \cos \Theta$$

Since $\mathbf{V_i} \perp \mathbf{V_j}$ then $\cos \Theta = \cos 90 = 0$

$$\mathbf{V_i} \perp \mathbf{V_j} = \text{Transpose}(\mathbf{V_i}) * \mathbf{V_j} = \mathbf{0}$$

Therefore, every pair of the eigenvector is $\perp$ to each other.

4. The eigenvector is a vector with the direction of maximum variance and eigenvalue is the number (scalar) which represent maximum variance.

5. Since eigenvectors of a covariance matrix are orthogonal (perpendicular) to each other, they provide a new set of axes (principal components) that are uncorrelated. This is beneficial because it means that the new features (principal components) are independent of one another, which simplifies the structure of the data and makes patterns more discernible.

That's all folks. You have completed the foundations required to learn PCA. I will see you again in the second part where we will start learning about mathematics and implementation of PCA.

UNIT: 3

Supervised Learning

Supervised Learning:

Supervised learning is the types of machine learning in which machines are trained using well “labelled” training data, and on basis of that data, machines predict the output. The labeled data means some input data is already tagged with the correct output.

In supervised learning, the training data provided to the machines work as the supervisor that teaches the machines to predict the output correctly. It applies the same concept as a student learns in the supervision of the teacher.

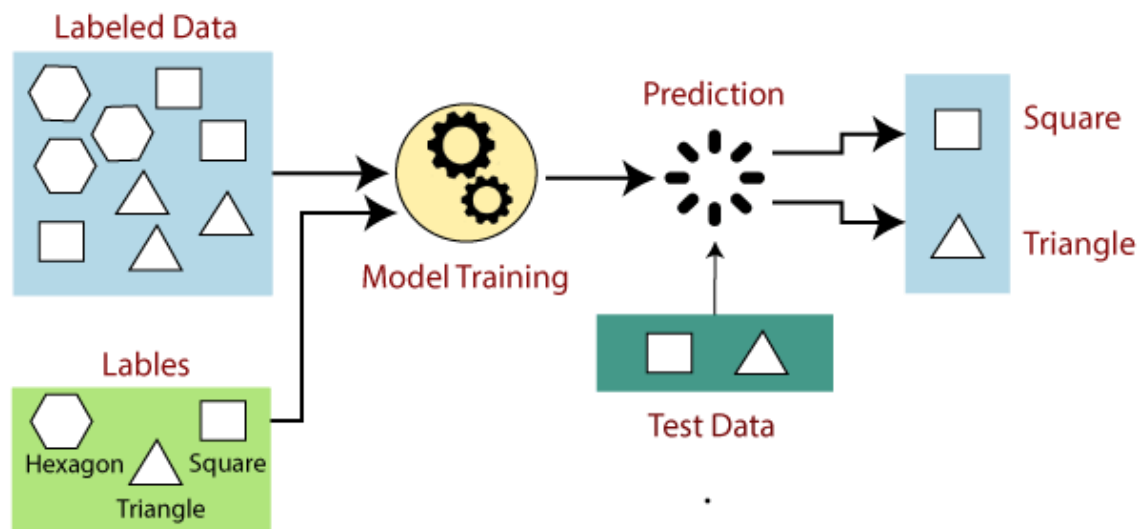
Supervised learning is a process of providing input data as well as correct output data to the machine learning model. The aim of a supervised learning is to **find a mapping function to map the input variable(x) with the output variable(y).**

In the real-world, supervised learning can be used for **Risk Assessment, Image classification, Fraud Detection, Spam filtering**, etc.

How Supervised Learning Works?

In supervised learning, models are trained using labelled dataset, where the model learns about each type of data. Once the training process is completed, the model is tested on the basis of test data (a subset of the training set), and then it predicts the output.

The working of Supervised learning can be easily understood by the below example and diagram:



Suppose we have a dataset of different types of shapes which includes square, rectangle, triangle, and Polygon. Now the first step is that we need to train the model for each shape.

- If the given shape has four sides, and all the sides are equal, then it will be labelled as a **Square**.
- If the given shape has three sides, then it will be labelled as a **triangle**.
- If the given shape has six equal sides then it will be labelled as **hexagon**.

Now, after training, we test our model using the test set, and the task of the model is to identify the shape.

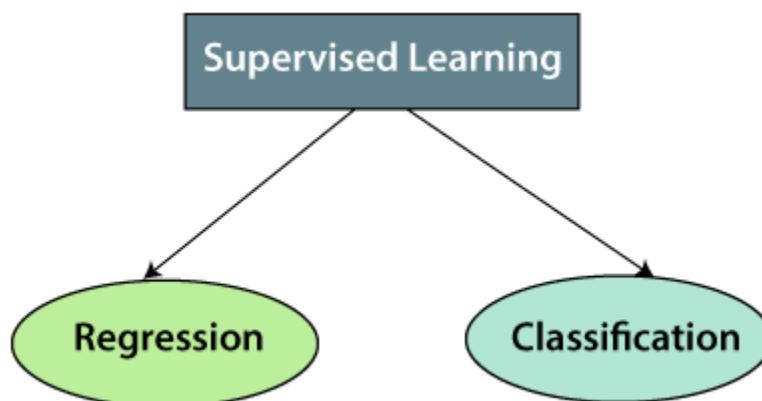
The machine is already trained on all types of shapes, and when it finds a new shape, it classifies the shape on the bases of a number of sides, and predicts the output.

Steps Involved in Supervised Learning:

- First Determine the type of training dataset
- Collect/Gather the labelled training data.
- Split the training dataset into training **dataset**, **test dataset**, and **validation dataset**.
- Determine the input features of the training dataset, which should have enough knowledge so that the model can accurately predict the output.
- Determine the suitable algorithm for the model, such as support vector machine, decision tree, etc.
- Execute the algorithm on the training dataset. Sometimes we need validation sets as the control parameters, which are the subset of training datasets.
- Evaluate the accuracy of the model by providing the test set. If the model predicts the correct output, which means our model is accurate.

Types of supervised Machine learning Algorithms:

Supervised learning can be further divided into two types of problems:



1. Regression

Regression algorithms are used if there is a relationship between the input variable and the output variable. It is used for the prediction of continuous variables, such as Weather forecasting, Market Trends, etc. Below are some popular Regression algorithms which come under supervised learning:

- Linear Regression
 - Regression Trees
 - Non-Linear Regression
 - Bayesian Linear Regression
 - Polynomial Regression
-

2. Classification

Classification algorithms are used when the output variable is categorical, which means there are two classes such as Yes-No, Male-Female, True-false, etc.

Spam Filtering,

- K-Nearest Neighbors
 - Naïve Bayes
 - Decision Trees
 - Logistic Regression
 - Support vector Machines
-

Advantages of Supervised learning:

- With the help of supervised learning, the model can predict the output on the basis of prior experiences.
 - In supervised learning, we can have an exact idea about the classes of objects.
 - Supervised learning model helps us to solve various real-world problems such as **fraud detection, spam filtering**, etc.
-

Disadvantages of supervised learning:

- Supervised learning models are not suitable for handling the complex tasks.
 - Supervised learning cannot predict the correct output if the test data is different from the training dataset.
 - Training required lots of computation times.
 - In supervised learning, we need enough knowledge about the classes of object.
-

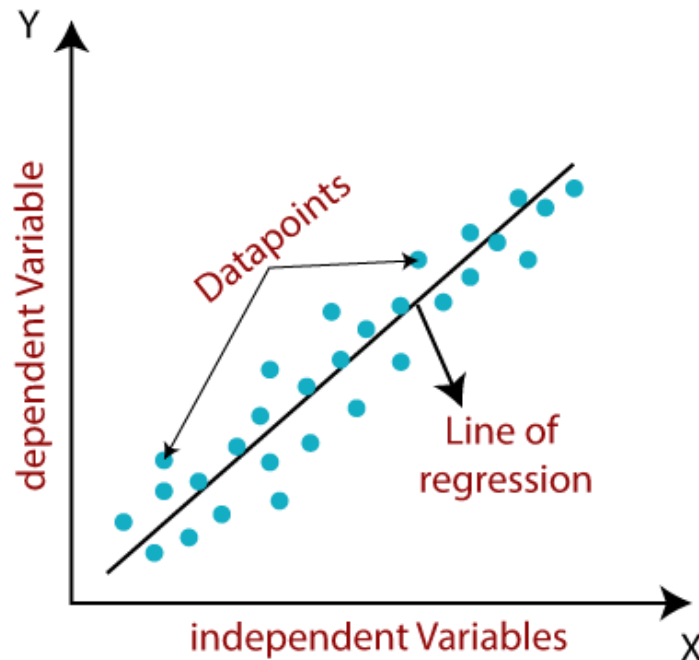
Linear Regression:

Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price**, etc.

Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (x) variables, hence called as linear regression. Since linear regression shows the linear

relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

The linear regression model provides a sloped straight line representing the relationship between the variables. Consider the below image:



Mathematically, we can represent a linear regression as:

$$y = a_0 + a_1x + \varepsilon$$

Here,

Y= Dependent Variable (Target Variable)

X= Independent Variable (predictor Variable)

a_0 = intercept of the line (Gives an additional degree of freedom)

a_1 = Linear regression coefficient (scale factor to each input value)

ε = random error

The values for x and y variables are training datasets for Linear Regression model representation.

Types of Linear Regression:

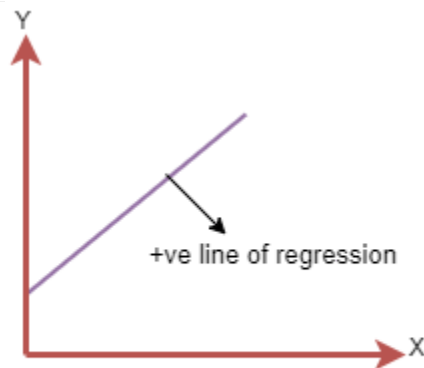
Linear regression can be further divided into two types of the algorithm:

- **Simple Linear Regression:** If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.
- **Multiple Linear Regression:** If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

Linear Regression Line:

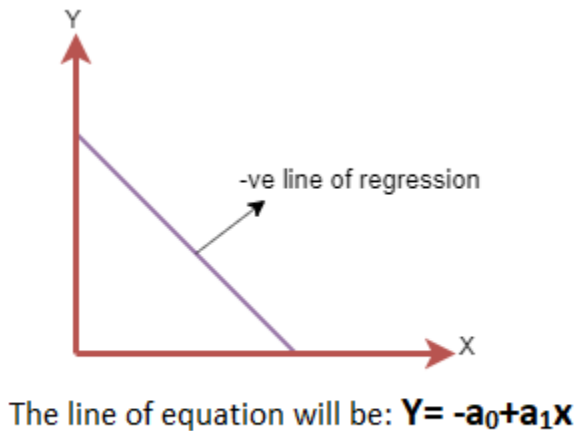
A linear line showing the relationship between the dependent and independent variables is called a **regression line**. A regression line can show two types of relationship:

- **Positive Linear Relationship:** If the dependent variable increases on the Y-axis and independent variable increases on X-axis, then such a relationship is termed as a Positive linear relationship.



The line equation will be: $Y = a_0 + a_1X$

- **Negative Linear Relationship:** If the dependent variable decreases on the Y-axis and independent variable increases on the X-axis, then such a relationship is called a negative linear relationship.



Finding the best fit line:

When working with linear regression, our main goal is to find the best fit line that means the error between predicted values and actual values should be minimized. The best fit line will have the least error.

The different values for weights or the coefficient of lines (a_0 , a_1) gives a different line of regression, so we need to calculate the best values for a_0 and a_1 to find the best fit line, so to calculate this we use cost function.

Cost function-

- The different values for weights or coefficient of lines (a_0 , a_1) gives the different line of regression, and the cost function is used to estimate the values of the coefficient for the best fit line.
- Cost function optimizes the regression coefficients or weights. It measures how a linear regression model is performing.
- We can use the cost function to find the accuracy of the **mapping function**, which maps the input variable to the output variable. This mapping function is also known as **Hypothesis function**.

For Linear Regression, we use the **Mean Squared Error (MSE)** cost function, which is the average of squared error occurred between the predicted values and actual values. It can be written as:

For the above linear equation, MSE can be calculated as:

$$MSE = \frac{1}{N} \sum_{i=1}^n (y_i - (a_1x_i + a_0))^2$$

Where,

N = Total number of observation

Y_i = Actual value

$(a_1x_i + a_0)$ = Predicted value

Residuals: The distance between the actual value and predicted values is called residual. If the observed points are far from the regression line, then the residual will be high, and so cost function will high. If the scatter points are close to the regression line, then the residual will be small and hence the cost function.

Gradient Descent:

- Gradient descent is used to minimize the MSE by calculating the gradient of the cost function.
- A regression model uses gradient descent to update the coefficients of the line by reducing the cost function.
- It is done by a random selection of values of coefficient and then iteratively update the values to reach the minimum cost function.

Model Performance:

The Goodness of fit determines how the line of regression fits the set of observations. The process of finding the best model out of various models is called optimization. It can be achieved by below method:

1. R-squared method:

- R-squared is a statistical method that determines the goodness of fit.
- It measures the strength of the relationship between the dependent and independent variables on a scale of 0-100%.
- The high value of R-square determines the less difference between the predicted values and actual values and hence represents a good model.
- It is also called a **coefficient of determination**, or **coefficient of multiple determination** for multiple regression.
- It can be calculated from the below formula:

$$\text{R-squared} = \frac{\text{Explained variation}}{\text{Total Variation}}$$

Assumptions of Linear Regression:

Below are some important assumptions of Linear Regression. These are some formal checks while building a Linear Regression model, which ensures to get the best possible result from the given dataset.

- **Linear relationship between the features and target:** Linear regression assumes the linear relationship between the dependent and independent variables.
- **Small or no multicollinearity between the features:** Multicollinearity means high-correlation between the independent variables. Due to multicollinearity, it may difficult to find the true

relationship between the predictors and target variables. Or we can say, it is difficult to determine which predictor variable is affecting the target variable and which is not. So, the model assumes either little or no multicollinearity between the features or independent variables.

- **Homoscedasticity Assumption:** Homoscedasticity is a situation when the error term is the same for all the values of independent variables. With homoscedasticity, there should be no clear pattern distribution of data in the scatter plot.
- **Normal distribution of error terms:** Linear regression assumes that the error term should follow the normal distribution pattern. If error terms are not normally distributed, then confidence intervals will become either too wide or too narrow, which may cause difficulties in finding coefficients. It can be checked using the **q-q plot**. If the plot shows a straight line without any deviation, which means the error is normally distributed.
- **No autocorrelations:** The linear regression model assumes no autocorrelation in error terms. If there will be any correlation in the error term, then it will drastically reduce the accuracy of the model. Autocorrelation usually occurs if there is a dependency between residual errors.

Linear Regression video: <https://youtu.be/zUQr6HAAKp4?si=y0vT6b6U3g1nHGFE>

K-Nearest Neighbor(KNN) Algorithm for Machine Learning

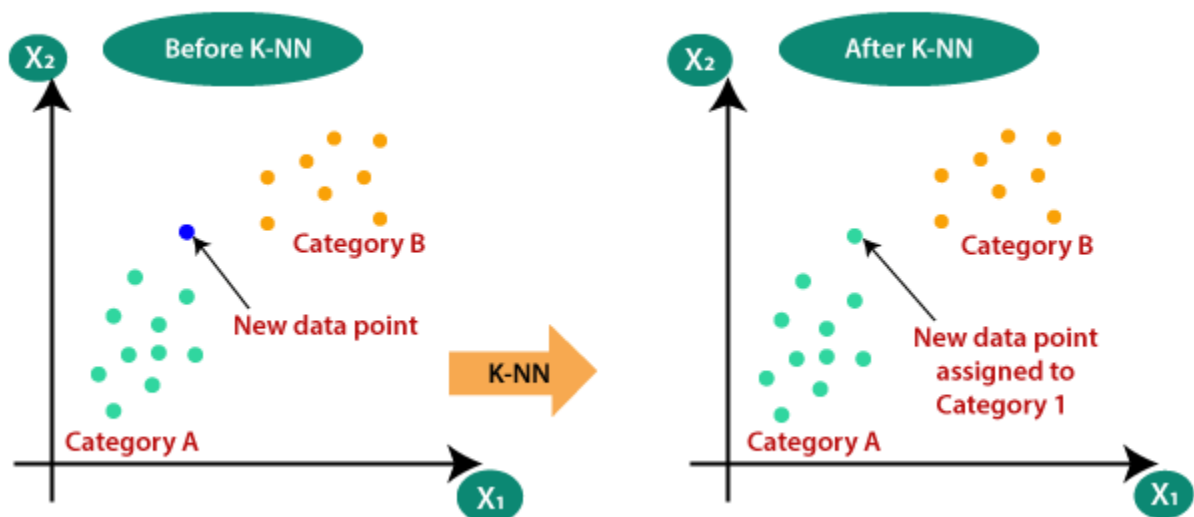
- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- **Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new

data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



Why do we need a K-NN Algorithm?

Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x_1 , so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:

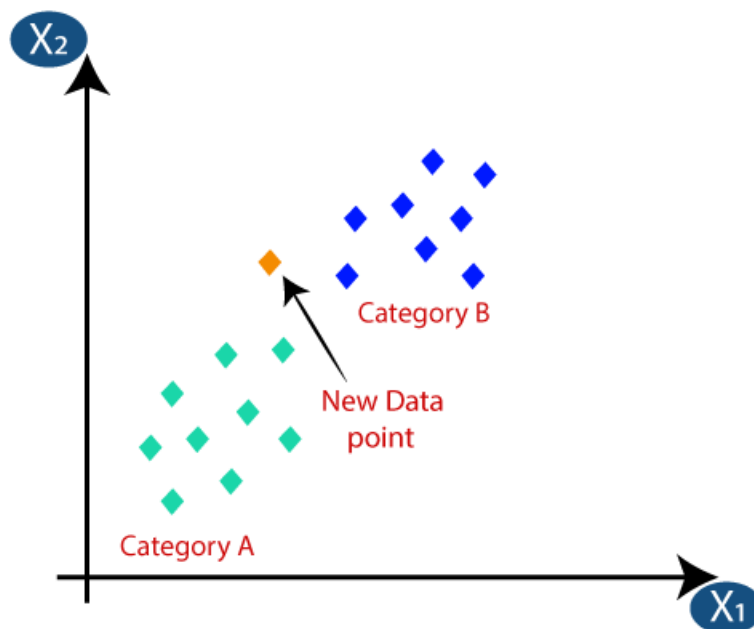


How does K-NN work?

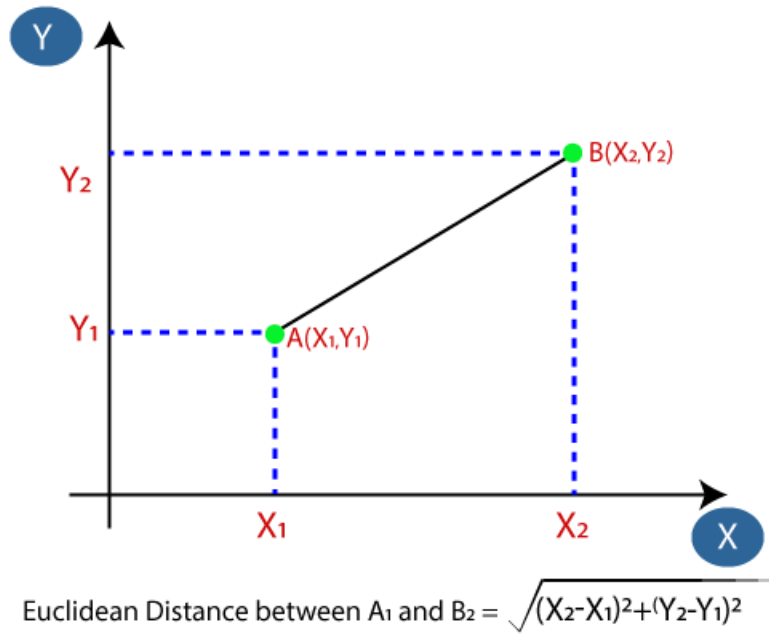
The K-NN working can be explained on the basis of the below algorithm:

- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.

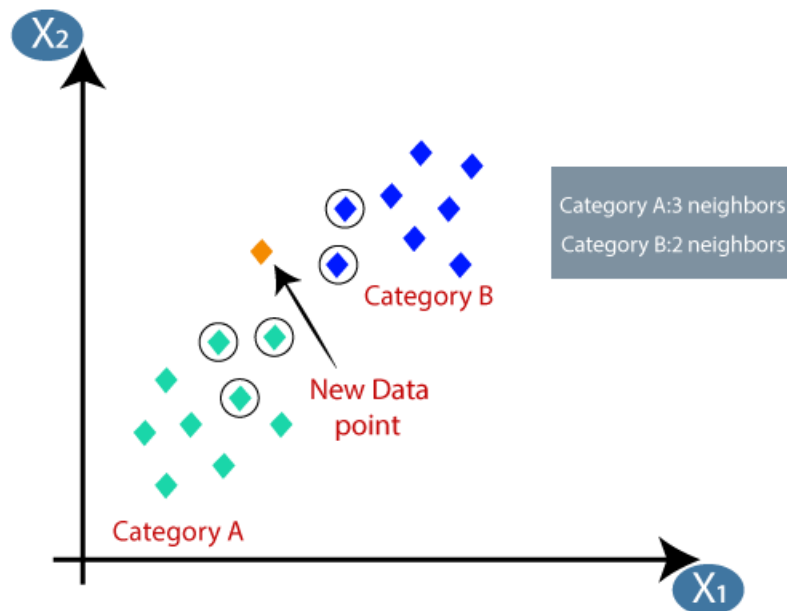
Suppose we have a new data point and we need to put it in the required category. Consider the below image:



- Firstly, we will choose the number of neighbors, so we will choose the $k=5$.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:



- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:



- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

How to select the value of K in the K-NN Algorithm?

Below are some points to remember while selecting the value of K in the K-NN algorithm:

- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
 - A very low value for K such as K=1 or K=2, can be noisy and lead to the effects of outliers in the model.
 - Large values for K are good, but it may find some difficulties.
-

Advantages of KNN Algorithm:

- It is simple to implement.
 - It is robust to the noisy training data
 - It can be more effective if the training data is large.
-

Disadvantages of KNN Algorithm:

- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.

KNN Algorithm Video: <https://youtu.be/O1nWXTXcCwI?si=EfeC-fzv96ioduxT>

Naïve Bayes Classifier Algorithm:

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on **Bayes theorem** and used for solving classification problems.
 - It is mainly used in *text classification* that includes a high-dimensional training dataset.
 - Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
 - **It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.**
 - Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis, and classifying articles.**
-

Why is it called Naïve Bayes?

The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, Which can be described as:

- **Naïve:** It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of

color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.

- **Bayes:** It is called Bayes because it depends on the principle of [Bayes' Theorem](#).

Bayes' Theorem:

- Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where,

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

P(A) is Prior Probability: Probability of hypothesis before observing the evidence.

P(B) is Marginal Probability: Probability of Evidence.

Working of Naïve Bayes' Classifier:

Working of Naïve Bayes' Classifier can be understood with the help of the below example:

Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**". So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions. So to solve this problem, we need to follow the below steps:

1. Convert the given dataset into frequency tables.
2. Generate Likelihood table by finding the probabilities of given features.
3. Now, use Bayes theorem to calculate the posterior probability.

Problem: If the weather is sunny, then the Player should play or not?

Solution: To solve this, first consider the below dataset:

	Outlook	Play
0	Rainy	Yes

1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2

Sunny	3	2
Total	10	5

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	$5/14=0.35$
Rainy	2	2	$4/14=0.29$
Sunny	2	3	$5/14=0.35$
All	$4/14=0.29$	$10/14=0.71$	

Applying Bayes'theorem:

$$P(\text{Yes}|\text{Sunny}) = P(\text{Sunny}|\text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$$

$$P(\text{Sunny}|\text{Yes}) = 3/10 = 0.3$$

$$P(\text{Sunny}) = 0.35$$

$$P(\text{Yes}) = 0.71$$

$$\text{So } P(\text{Yes}|\text{Sunny}) = 0.3 * 0.71 / 0.35 = \mathbf{0.60}$$

$$P(\text{No}|\text{Sunny}) = P(\text{Sunny}|\text{No}) * P(\text{No}) / P(\text{Sunny})$$

$$P(\text{Sunny}|\text{NO}) = 2/4 = 0.5$$

$$P(\text{No}) = 0.29$$

$$P(\text{Sunny}) = 0.35$$

$$\text{So } P(\text{No}|\text{Sunny}) = 0.5 * 0.29 / 0.35 = \mathbf{0.41}$$

So as we can see from the above calculation that $P(\text{Yes}|\text{Sunny}) > P(\text{No}|\text{Sunny})$

Hence on a Sunny day, Player can play the game.

Advantages of Naïve Bayes Classifier:

- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
 - It can be used for Binary as well as Multi-class Classifications.
 - It performs well in Multi-class predictions as compared to the other Algorithms.
 - It is the most popular choice for **text classification problems**.
-

Disadvantages of Naïve Bayes Classifier:

- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.
-

Applications of Naïve Bayes Classifier:

- It is used for **Credit Scoring**.
 - It is used in **medical data classification**.
 - It can be used in **real-time predictions** because Naïve Bayes Classifier is an eager learner.
 - It is used in Text classification such as **Spam filtering** and **Sentiment analysis**.
-

Types of Naïve Bayes Model:

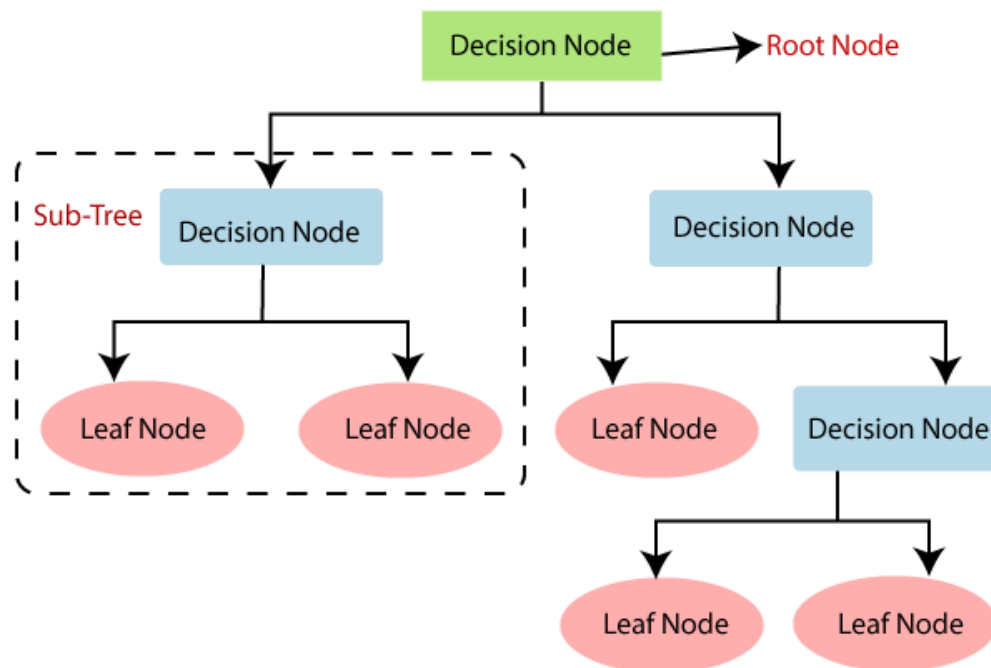
There are three types of Naive Bayes Model, which are given below:

- **Gaussian:** The Gaussian model assumes that features follow a normal distribution. This means if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- **Multinomial:** The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed. It is primarily used for document classification problems, it means a particular document belongs to which category such as Sports, Politics, education, etc. The classifier uses the frequency of words for the predictors.
- **Bernoulli:** The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables. Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.

Naïve Bayes Video: <https://youtu.be/GBMMtXRiQX0?si=K-LGj1n4Cqio3dTD>

Decision Tree Classification Algorithm

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome**.
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- Below diagram explains the general structure of a decision tree:



Why use Decision Trees?

There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

□ **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.

- **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- **Branch/Sub Tree:** A tree formed by splitting the tree.
- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.
- **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

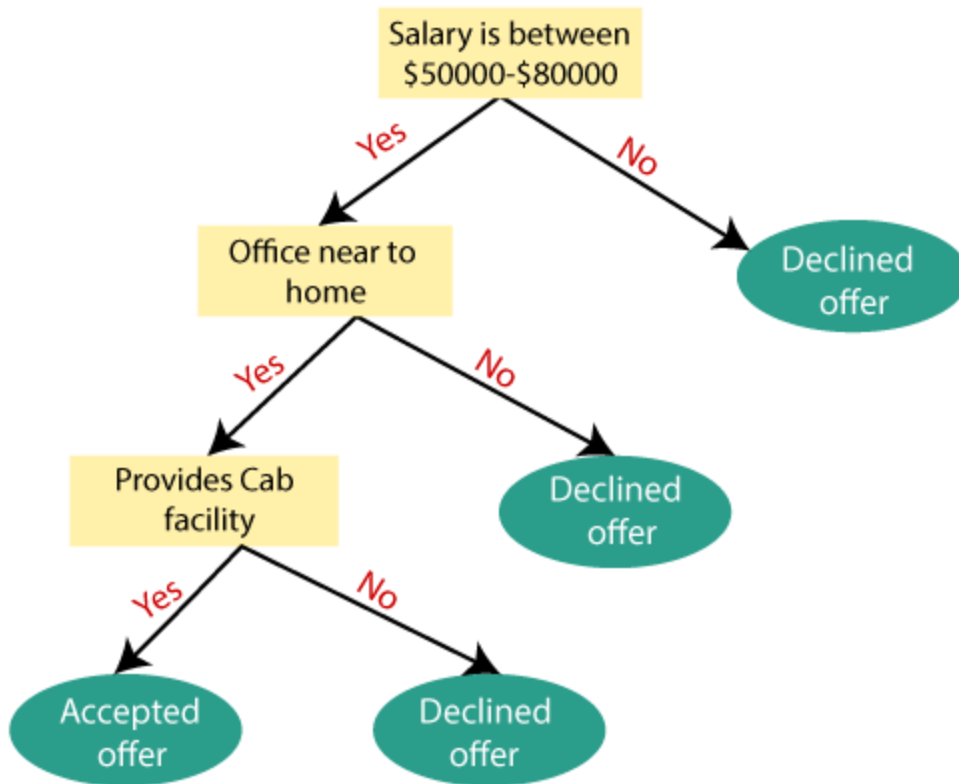
How does the Decision Tree algorithm Work?

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
 - **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
 - **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
 - **Step-4:** Generate the decision tree node, which contains the best attribute.
 - **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.
-

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:



Attribute Selection Measures

While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM**. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:

- **Information Gain**
- **Gini Index**

1. Information Gain:

- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)]

Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$Entropy(s) = -P(yes)\log_2 P(yes) - P(no) \log_2 P(no)$$

Where,

- **S= Total number of samples**
 - **P(yes)= probability of yes**
 - **P(no)= probability of no**
-

2. Gini Index:

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
 - An attribute with the low Gini index should be preferred as compared to the high Gini index.
 - It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
 - Gini index can be calculated using the below formula:
-

$$\text{Gini Index} = 1 - \sum_i P_i^2$$

Pruning: Getting an Optimal Decision tree

Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.

A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset. Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning. There are mainly two types of tree **pruning** technology used:

- **Cost Complexity Pruning**
 - **Reduced Error Pruning.**
-

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
 - It can be very useful for solving decision-related problems.
 - It helps to think about all the possible outcomes for a problem.
 - There is less requirement of data cleaning compared to other algorithms.
-

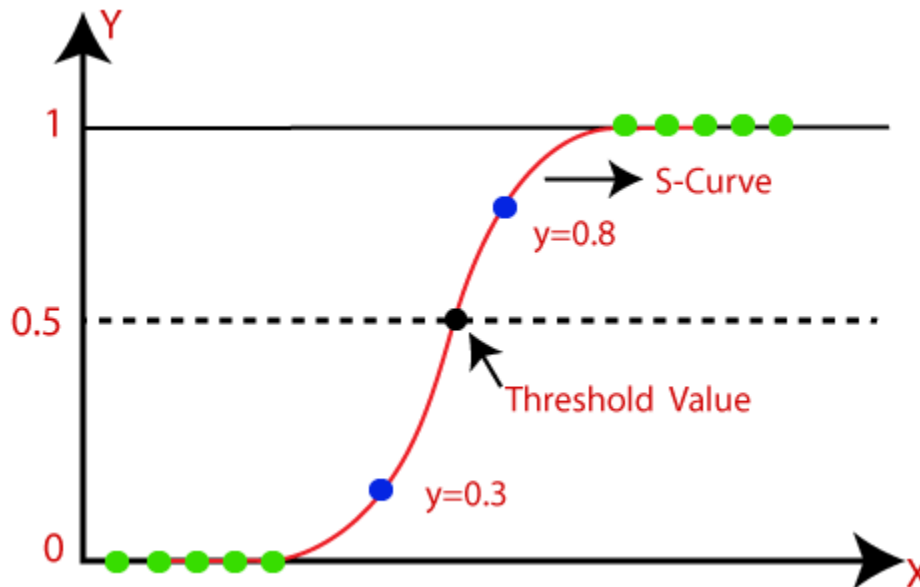
Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm**.
- For more class labels, the computational complexity of the decision tree may increase.

Decision Tree Video: https://youtu.be/mvveVcbHynE?si=8-Smt_X2Gfdf2lPg

Logistic Regression in Machine Learning

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**
- Logistic Regression is much similar to the Linear Regression except that how they are used. Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems.**
- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification. The below image is showing the logistic function:



Note: Logistic regression uses the concept of predictive modeling as regression; therefore, it is called logistic regression, but is used to classify samples; Therefore, it falls under the classification algorithm.

Logistic Function (Sigmoid Function):

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

Assumptions for Logistic Regression:

- The dependent variable must be categorical in nature.
- The independent variable should not have multi-collinearity.

Logistic Regression Equation:

The Logistic regression equation can be obtained from the Linear Regression equation. The mathematical steps to get Logistic Regression equations are given below:

- We know the equation of the straight line can be written as:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

- In Logistic Regression y can be between 0 and 1 only, so for this let's divide the above equation by (1-y):

$$\frac{y}{1-y}; 0 \text{ for } y=0, \text{ and infinity for } y=1$$

- But we need range between -[infinity] to +[infinity], then take logarithm of the equation it will become:

$$\log \left[\frac{y}{1-y} \right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

The above equation is the final equation for Logistic Regression.

Type of Logistic Regression:

On the basis of the categories, Logistic Regression can be classified into three types:

- **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
- **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"
- **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

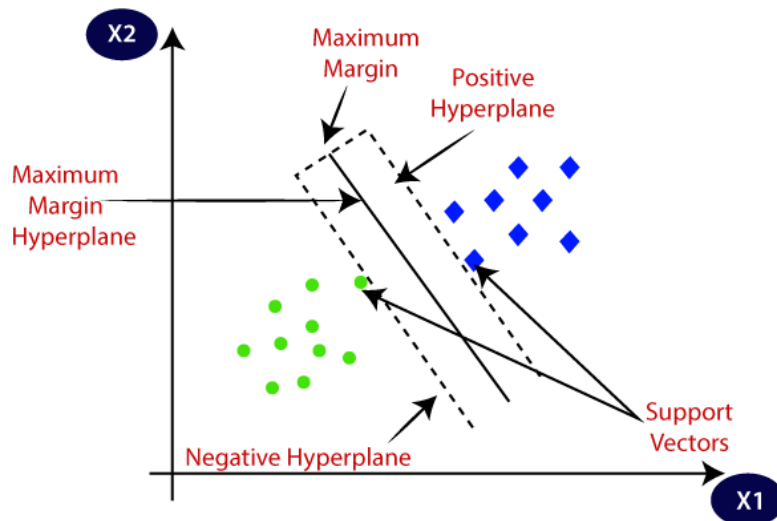
Logistic regression video: <https://youtu.be/r8OjlgWpAI0?si=xg58J3HrZUKia9hH>

Support Vector Machine Algorithm

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

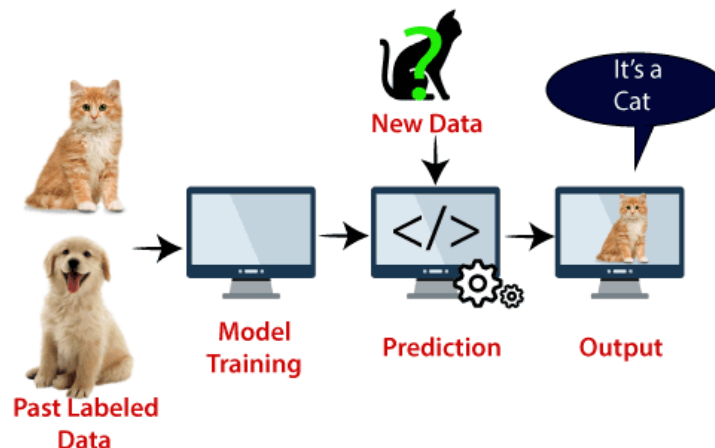
The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



1.

Example: SVM can be understood with the example that we have used in the KNN classifier. Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat. Consider the below diagram:



SVM algorithm can be used for **Face detection, image classification, text categorization**, etc.

Types of SVM

SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
 - **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.
-

Hyperplane and Support Vectors in the SVM algorithm:

Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.

We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

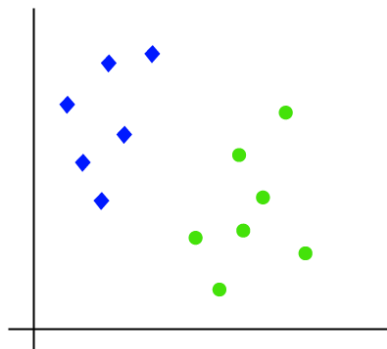
Support Vectors:

The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

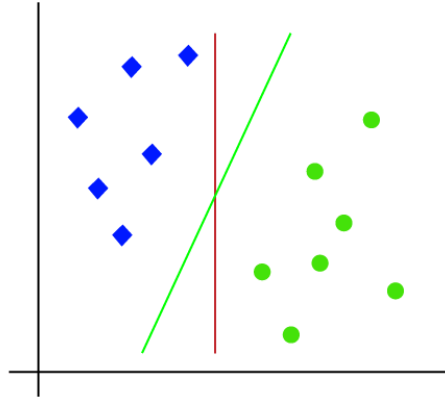
How does SVM works?

Linear SVM:

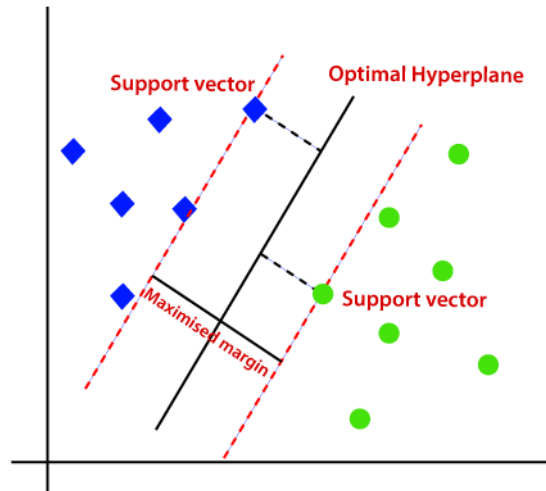
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:

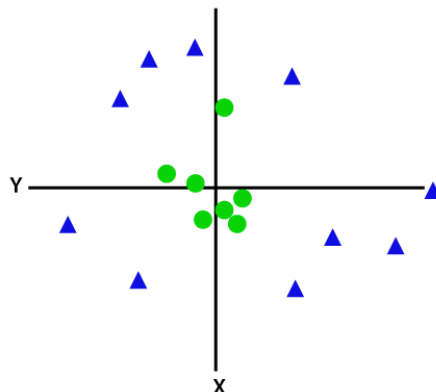


Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



Non-Linear SVM:

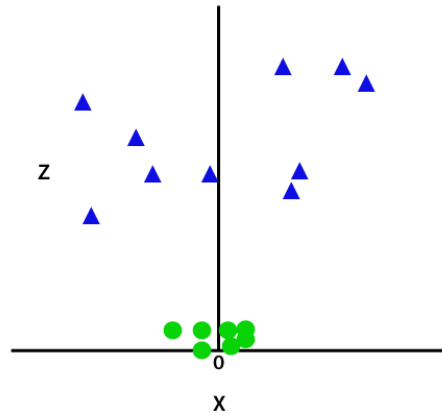
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



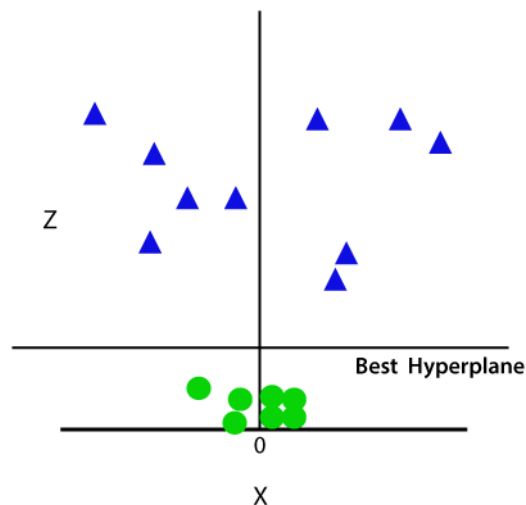
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$$z = x^2 + y^2$$

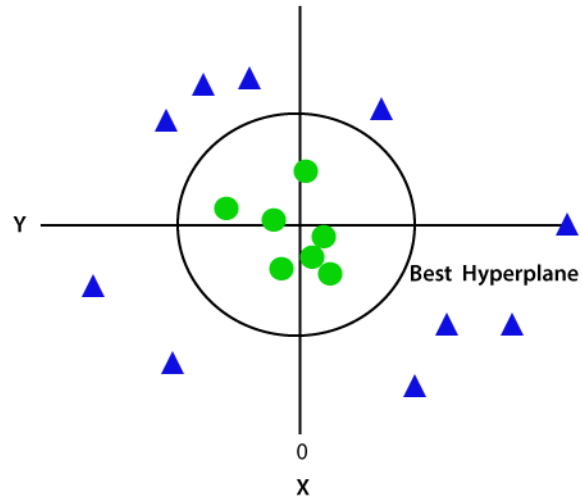
By adding the third dimension, the sample space will become as below image:



So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with $z=1$, then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.

Support Vector Video: <https://youtu.be/NDqACjz5j8g?si=UJ7LUuUApAn6HecK>

UNIT: 4

Unsupervised Learning

What is Unsupervised Learning?

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things. It can be defined as:

Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.

Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format.**

Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.



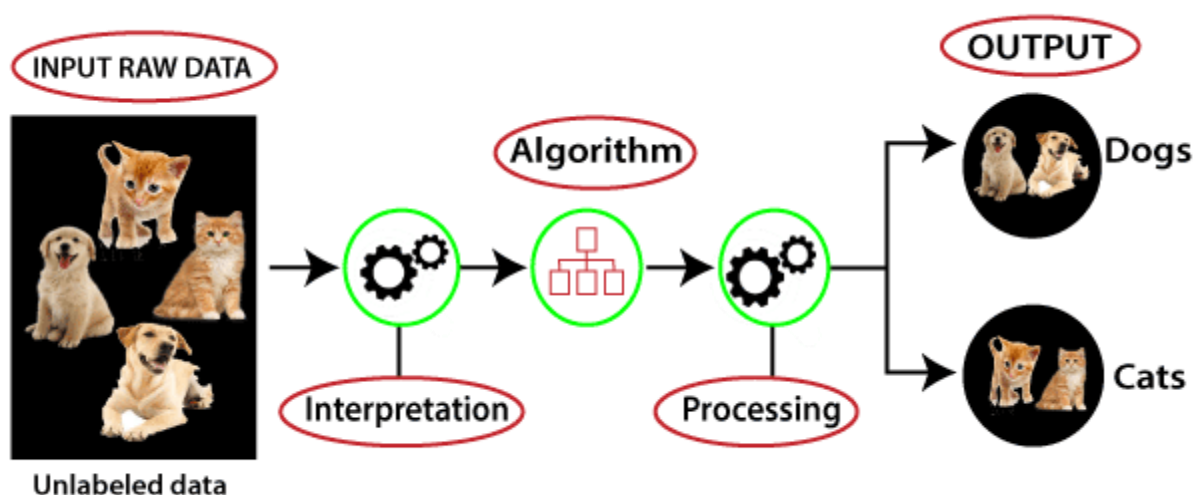
Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

Working of Unsupervised Learning

Working of unsupervised learning can be understood by the below diagram:

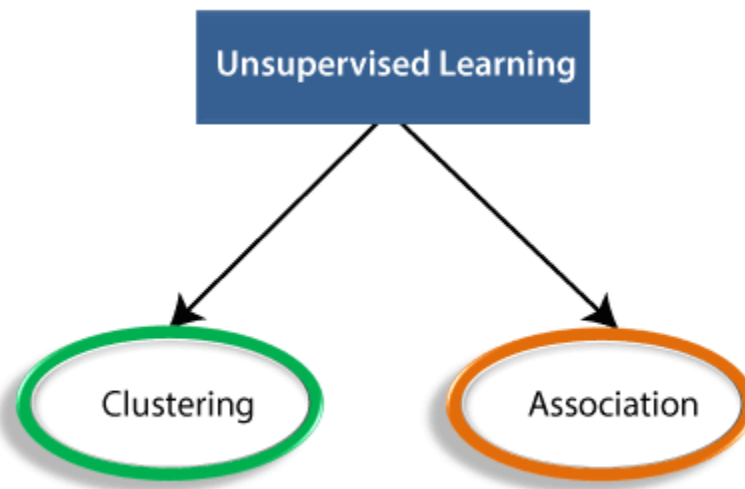


Here, we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabeled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.

Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:



- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remain into a group and have less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.
 - **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.
-

Unsupervised Learning algorithms:

Below is the list of some popular unsupervised learning algorithms:

- **K-means clustering**
 - **KNN (k-nearest neighbors)**
 - **Hierarchical clustering**
 - **Anomaly detection**
 - **Neural Networks**
 - **Principle Component Analysis**
 - **Independent Component Analysis**
 - **Apriori algorithm**
 - **Singular value decomposition**
-

Advantages of Unsupervised Learning

- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
 - Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled data.
-

Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.

Clustering:

The clustering methods are broadly divided into **Hard clustering** (datapoint belongs to only one group) and **Soft Clustering** (data points can belong to another group also). But there are also other various approaches of Clustering exist. Below are the main clustering methods used in Machine learning:

1. **Partitioning Clustering**
2. **Density-Based Clustering**
3. **Distribution Model-Based Clustering**
4. **Hierarchical Clustering**
5. **Fuzzy Clustering**

Here we are discussing mainly popular Clustering algorithms that are widely used in machine learning:

1. **K-Means algorithm**
2. **Mean-shift algorithm**
3. **DBSCAN Algorithm:** It stands for **Density-Based Spatial Clustering of Applications with Noise**. It
4. **Expectation-Maximization Clustering using GM**
5. **Agglomerative Hierarchical algorithm**
6. **Affinity Propagation**

K-Means Clustering Algorithm

K-Means Clustering is an [Unsupervised Learning algorithm](#), which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if K=2, there will be two clusters, and for K=3, there will be three clusters, and so on.

It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.

It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.

It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.

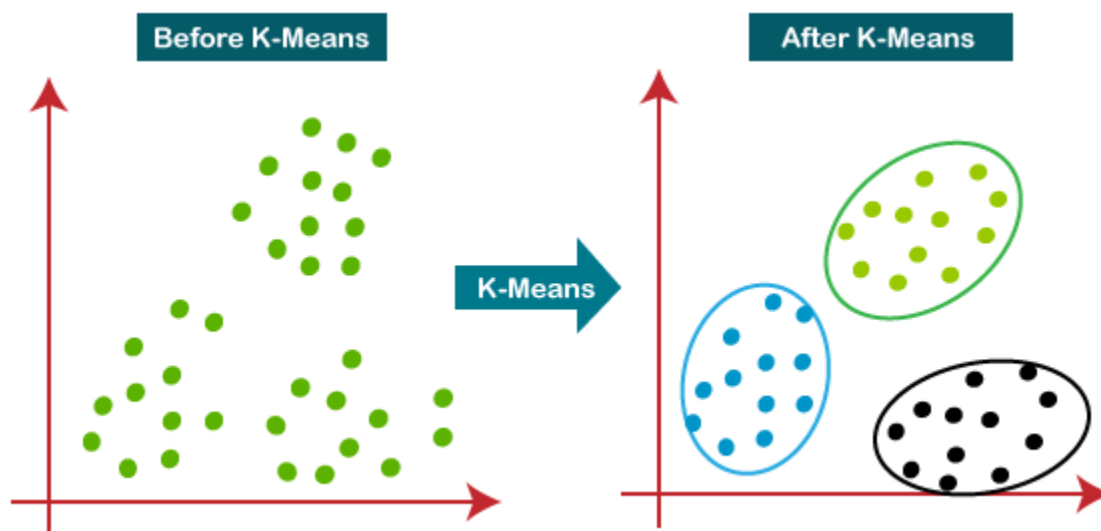
The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm.

The k-means clustering algorithm mainly performs two tasks:

- Determines the best value for K center points or centroids by an iterative process.
- Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.

Hence each cluster has datapoints with some commonalities, and it is away from other clusters.

The below diagram explains the working of the K-means Clustering Algorithm:



How does the K-Means Algorithm Work?

The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

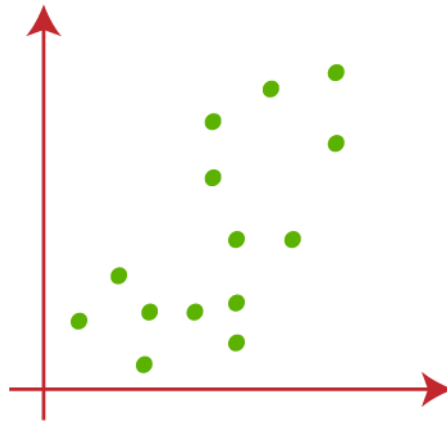
Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

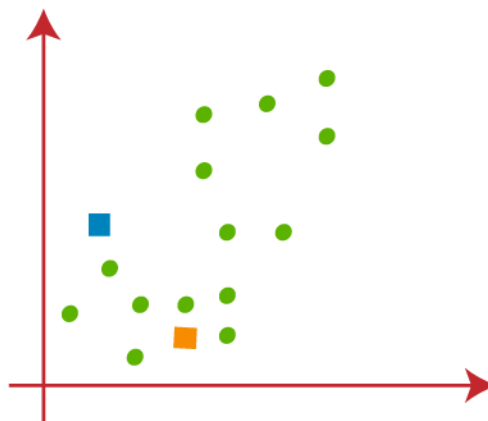
Step-7: The model is ready.

Let's understand the above steps by considering the visual plots:

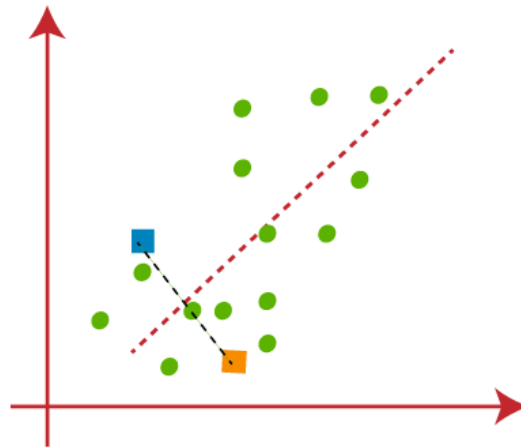
Suppose we have two variables M1 and M2. The x-y axis scatter plot of these two variables is given below:



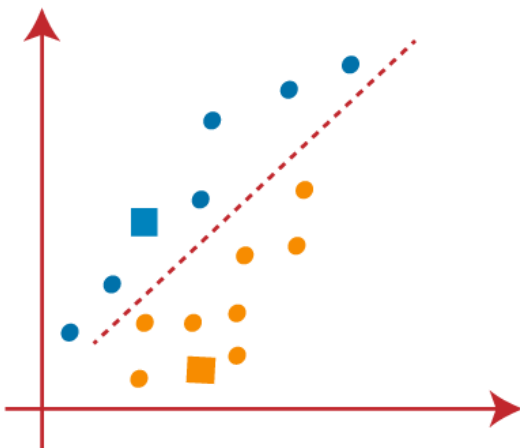
- Let's take number k of clusters, i.e., $K=2$, to identify the dataset and to put them into different clusters. It means here we will try to group these datasets into two different clusters.
- We need to choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point. So, here we are selecting the below two points as k points, which are not the part of our dataset. Consider the below image:



- Now we will assign each data point of the scatter plot to its closest K-point or centroid. We will compute it by applying some mathematics that we have studied to calculate the distance between two points. So, we will draw a median between both the centroids. Consider the below image:
-

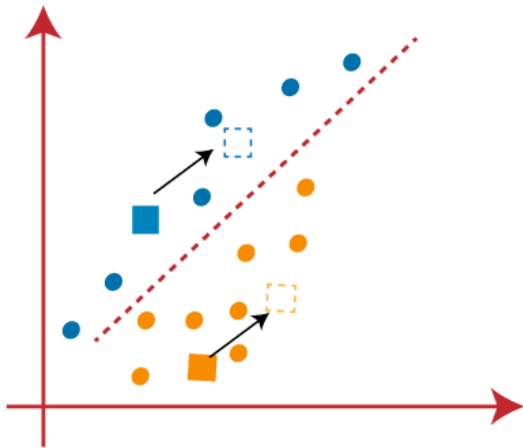


From the above image, it is clear that points left side of the line is near to the K1 or blue centroid, and points to the right of the line are close to the yellow centroid. Let's color them as blue and yellow for clear visualization.

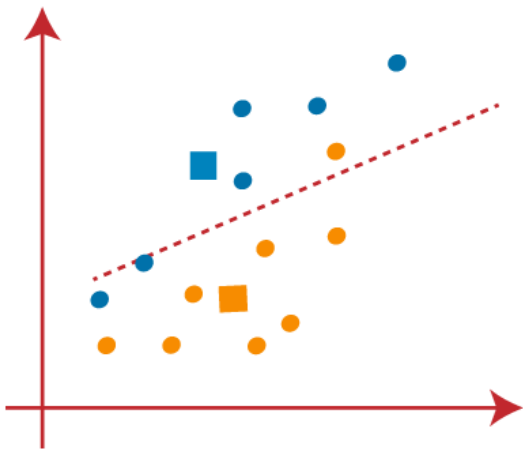


- As we need to find the closest cluster, so we will repeat the process by choosing a **new centroid**. To choose the new centroids, we will compute the center of gravity of these centroids, and will find new centroids as

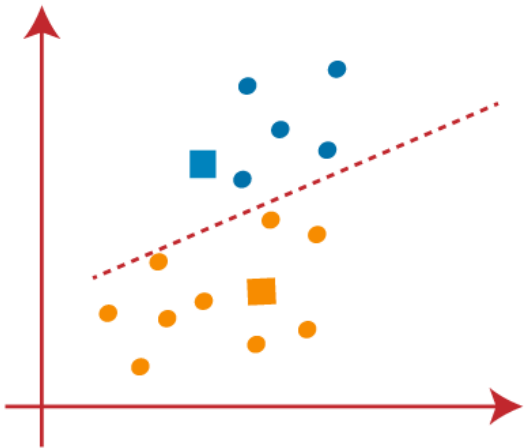
below:



- Next, we will reassign each datapoint to the new centroid. For this, we will repeat the same process of finding a median line. The median will be like below image:

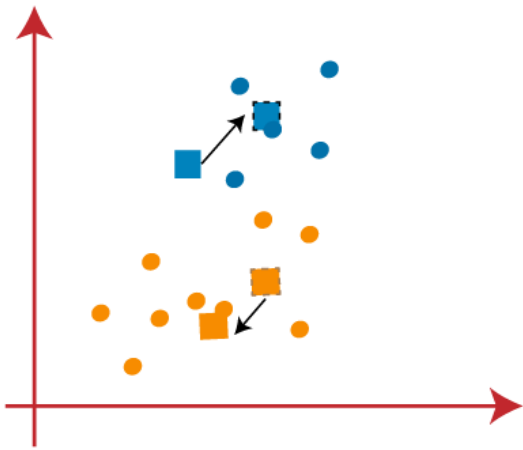


From the above image, we can see, one yellow point is on the left side of the line, and two blue points are right to the line. So, these three points will be assigned to new centroids.

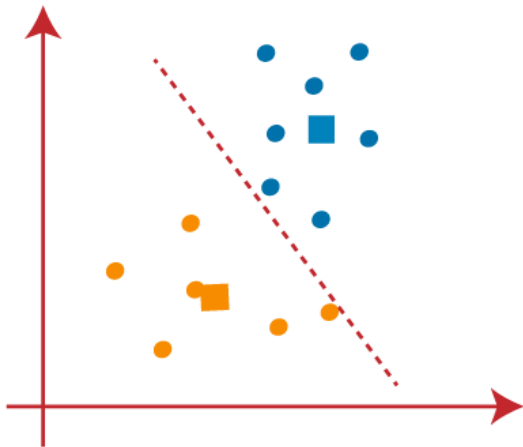


As reassignment has taken place, so we will again go to the step-4, which is finding new centroids or K-points.

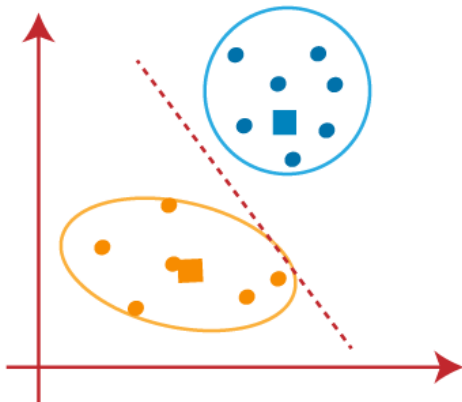
- We will repeat the process by finding the center of gravity of centroids, so the new centroids will be as shown in the below image:



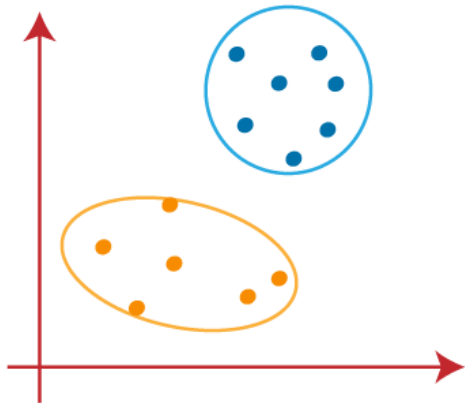
- As we got the new centroids so again will draw the median line and reassign the data points. So, the image will be:



- We can see in the above image; there are no dissimilar data points on either side of the line, which means our model is formed. Consider the below image:



As our model is ready, so we can now remove the assumed centroids, and the two final clusters will be as shown in the below image:



How to choose the value of "K number of clusters" in K-means Clustering?

The performance of the K-means clustering algorithm depends upon highly efficient clusters that it forms. But choosing the optimal number of clusters is a big task. There are some different ways to find the optimal number of clusters, but here we are discussing the most appropriate method to find the number of clusters or value of K. The method is given below:

Elbow Method

The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS value. **WCSS** stands for **Within Cluster Sum of Squares**, which defines the total variations within a cluster. The formula to calculate the value of WCSS (for 3 clusters) is given below:

$$WCSS = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i, C_2)^2 + \sum_{P_i \text{ in Cluster3}} \text{distance}(P_i, C_3)^2$$

In the above formula of WCSS,

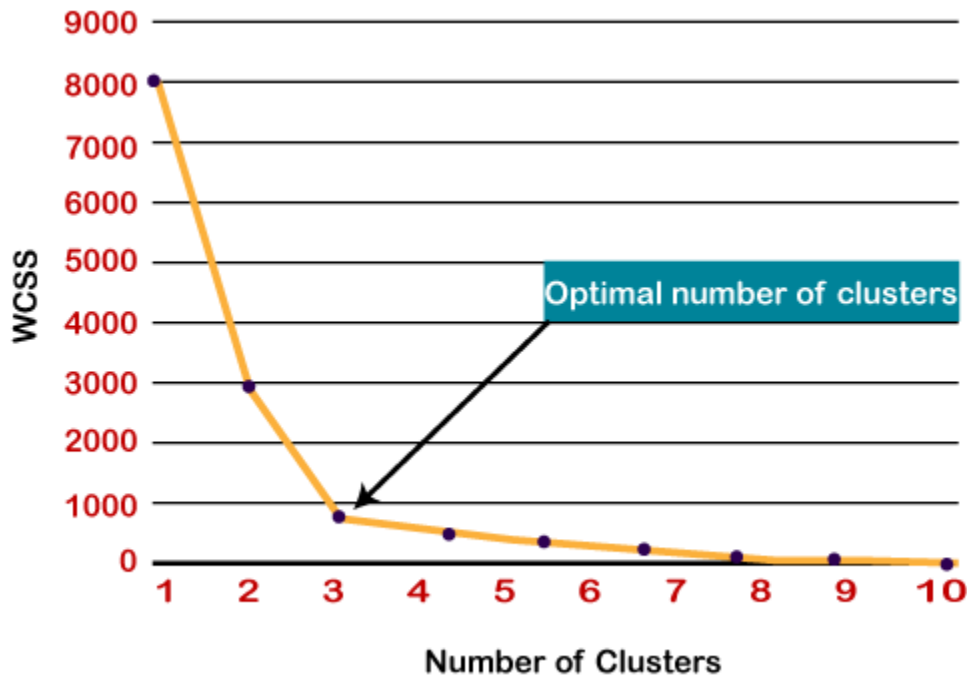
$\sum_{P_i \text{ in Cluster1}} \text{distance}(P_i, C_1)^2$: It is the sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the other two terms.

To measure the distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance.

To find the optimal value of clusters, the elbow method follows the below steps:

- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the WCSS value.
- Plots a curve between calculated WCSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K.

Since the graph shows the sharp bend, which looks like an elbow, hence it is known as the elbow method. The graph for the elbow method looks like the below image:



Note: We can choose the number of clusters equal to the given data points. If we choose the number of clusters equal to the data points, then the value of WCSS becomes zero, and that will be the endpoint of the plot.

What Is Ensemble Learning?

Ensemble learning refers to a machine learning approach where several models are trained to address a common problem, and their predictions are combined to enhance the overall performance. The idea behind ensemble learning is that by combining multiple models, each with its strengths and weaknesses, the ensemble can achieve better results than any single model alone. Ensemble learning can be applied to various machine learning tasks, including classification, regression, and clustering. Some common ensemble learning methods include bagging, boosting, and [stacking](#).

Ensemble Techniques:

Ensemble techniques in machine learning involve combining multiple models to improve performance. One common ensemble technique is bagging, which uses bootstrap sampling to create multiple datasets from the original data and trains a model on each [dataset](#). Another technique is boosting, which trains models sequentially, each focusing on the previous models' mistakes. Random forests are a popular ensemble method that uses decision trees as base learners and combines their predictions to make a final prediction. Ensemble techniques are effective because they reduce overfitting and improve generalization, leading to more robust models.

Simple Ensemble Techniques

Max Voting

Averaging

Weighted Averaging

Advanced Ensemble Techniques:

Stacking

Blending

Bagging (Bootstrap Aggregating)

Boosting

Bagging and Boosting Algorithms

Random Forest

Bagged Decision Trees